1. Supporting the SAFE Initiative 2
1.1 Energy Delivery Session Identifier (EDSI) 7
1.2 Session Signature 8
2. Networking Interfaces 9
2.1 Charger to DC Meter Communications 10
2.2 Cryptographic Security 11
2.2.1 Entropy source 12
2.2.2 Random numbers 18
3. External Non Volatile Storage 19
3.1 Audit Event Log 26
3.2 Charge Session Log 29
3.3 Application Package Manager 30
4. Supervisory State Machine 36
4.1 Self Tests 38
5. DC Meter Configuration 41
6. Time on the DC Meter 44
7. Trusted Technician Interface (TTI) 45

Supporting the SAFE Initiative

Table of Contents

- SAFE Initiative Introduction
  - References
  - Fundamentals
- Solution
  - Example OCMF message with added comments
  - Example SAFE packet

Table of Figures

- Figure – Sequence diagram of order of events to receive and verify a bill.
- Figure – Diagram showing information flow between objects for a user receiving a session's bill.

SAFE Initiative Introduction

S.A.F.E stands for Software Alliance For E-mobility.

Transparenzsoftware has several supported formats for transmitting metering information:

1. Open Charge Metering Format (OCMF)
2. ASFEN
3. EDL_40_P
4. EDL_40_SIG

Tritium's DC Meter supports only OCMF and can be verified using the Transparenzsoftware.

References

ID	Title	Source	Notes
SSI1	Specification Fileformat Transparency-Software V2 KLMA 2018-07-10.pdf	Z:\Tritium\TRI110 - DC Meter\00 - Documents\TRI110.013 SAFE initiative\HTB file format info sent by HTB CEO 180725\	Format of information including charge session details to be sent from the DC Metering system to has.to.be transparency software.
SSI2	OCMF specification	http://hers.abl.de/SAFE/Datenformat_OCMF/Datenformat_OCMF_v1.0.pdf Z:\Tritium\TRI110 - DC Meter\00 - Documents\TRI110.013 SAFE initiative\supported formats\Datenformat_OCMF_v1.0.pdf	There is a google translate version at the location specified in "Source": "Datenformat_OCMF_v1.0-Google Translate.htm"

Fundamentals

1. Contents of the metering payload contains a cryptographic signature from the meter to ensure authenticity of the information, so data can be in easy to read plain text.
2. The user can search a CPO for their charge session to retrieve a previous metering payload.
3. The user can obtain the meter/charger public key from a trusted source (government server or TSO display).
4. The charge session details are retrieved from the portal (CPO, Tritium, has.to.be/Transparency, government agency, etc) that recorded it.
5. The user can verify the data with another application (such as a combination of grep and OpenSSL command line).

Solution

In order to be as interoperable as possible, the DC Meter will transmit measured data in the format specified in SSI2. This provides the most flexibility by supporting the SAFE initiative and supporting a mechanism that doesn't require the has.to.be software.

1. Uses the DC Meter as a storage device.
2. Uses the signing of information for authenticity.
3. Uses the retrieval of the meter's public key from a trusted authority to validate signed data.
4. The DC Meter will store the information for 3 months.
5. The Provider will supply the information when requested.
6. The "User" can validate using other independent 3rd party signature validation tools such as OpenSSL via command line arguments.

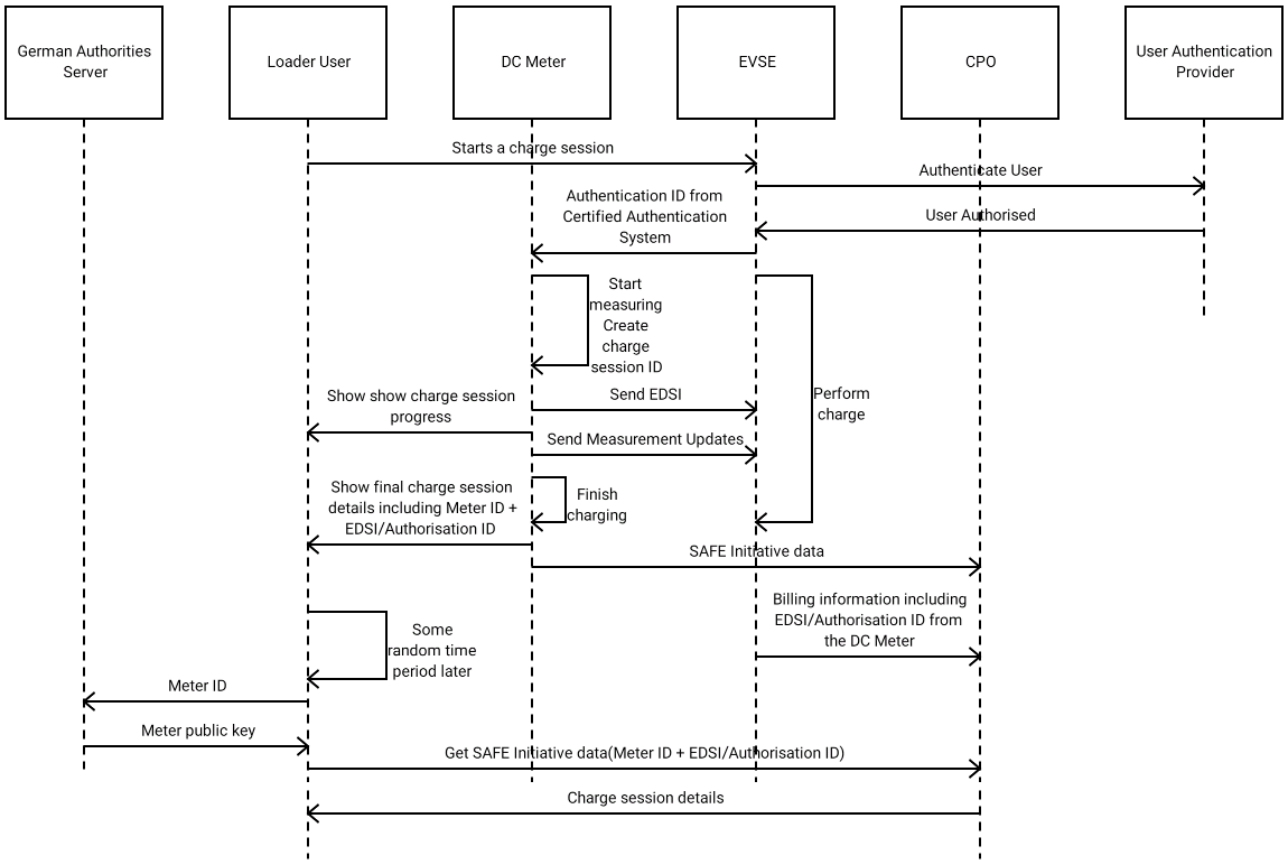


Figure – Sequence diagram of order of events to receive and verify a bill.

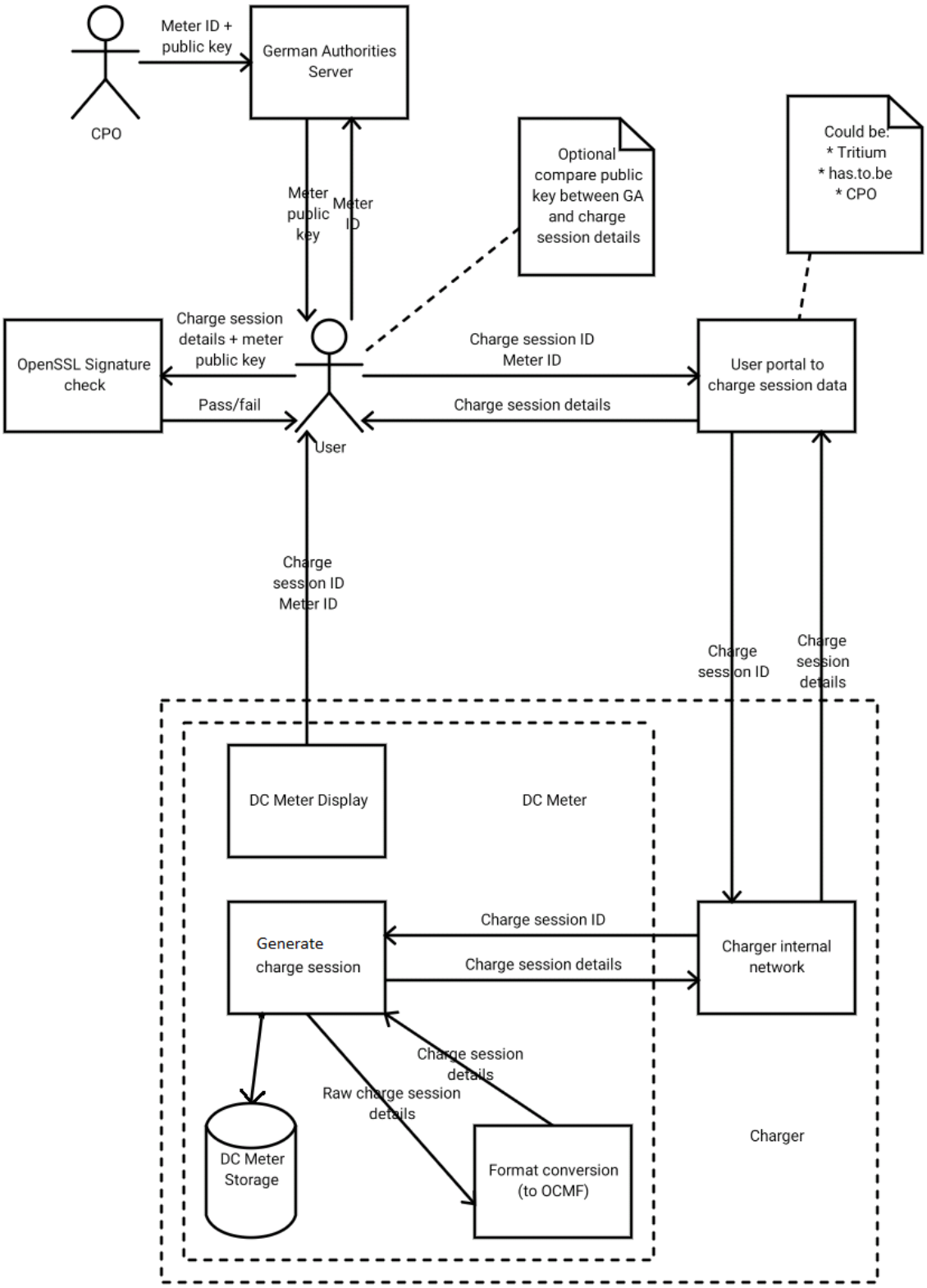


Figure – Diagram showing information flow between objects for a user receiving a session's bill.

Example OCMF message with added comments

The DC Meter complies with the OCMF v1.0 (Open Charge Metering Format) which can be obtained here:

http://hers.abl.de/SAFE/Datenformat_OCMF/Datenformat_OCMF_v1.0.pdf

To support the flexibility of external billing arrangements, reading entries are initally created every 15 minutes of charging with energy reading value. The end of transaction reading entry will contain the total session energy usage.

Due to memory constraints the 15 minute logging interval will continue for 6 hours, then reduce in rate to one reading entry per hour. After 48 hours of charging, the remainder of the charge session will be logged to the final reading entry, and no further reading entries will be created.

Tritium's DC Meter makes use of two optional manufacturer extension fields:

- VS - in user assignment portion of main data - contains the 'Session Signature', a globally unique identifier for the charge session, generated by the DC Meter.
- UT - contains the name of the tariff active in this charge (max 16 characters).

```
OCMF | {
        "FV": "1.0",                                                  // Protocol version number
        "GI": "Tritium Pty Ltd",                           // Gateway Identification: Manufacturer - same as MV
        "GS": "DCM202000001",                                  // Gateway serial - same as MS
        "PG": "T0000000010",                                  // Session ID (2^32) - Energy Delivery Session ID
        "MV": "Tritium Pty Ltd",                           // Meter vendor
        "MS": "DCM202000001",                                  // Meter serial ID (mandatory)
        "MF": "1.011",                                                 // Meter firmware
        "IS": "false",                                                 // Identification status of user
        "IL": "HEARSAY",                                            // Identification level
        "IF": ["RFID_NONE", "OCPP_NONE"],        // Identification flags
        "IT": "LOCAL",                                                 // Identification Type
        "ID": "1F2D3A4F5506C7",                                  // Identification Data - Charger Session ID
        "VS": "BAF9YAATLR68",                                  // Tritium special: Space for charge session
signature generated by DC Meter

        "RD" : [
                {"TM":"2021-09-15T00:50:57,000+0200 S","TX":"B","RV":"0.000","RU":"kWh","ST":"G","RI":"1-b:1.8.
e","UT":"Default Tariff"},
                {"TM":"2021-09-15T00:51:41,000+0200 S","TX":"C","RV":"3.959","RU":"kWh","ST":"G","RI":"1-b:1.8.
e","UT":"Default Tariff"},
                {"TM":"2021-09-15T00:51:41,000+0200 S","TX":"E","RV":"3.959","RU":"kWh","ST":"G","RI":"1-b:1.8.
e","UT":"Default Tariff"}
                ]
        } | {
                "SD":"
3044022079C63D04EA437654780D417063C6AFB4D6B61AEF50EEB3396EA5BA6E6DA792DE022008572037339682021 3B500ED637889F40220
5CE1C586B69AA95E8ED3F24537A5"
        }
```

Example SAFE packet

```
<?xml version="1.0" encoding="UTF-8"?>
<values>
        <value>
                <signedData format="OCMF" encoding="plain">OCMF|{"FV":"1.0","GI":"Tritium Pty Ltd","GS":"123","
PG":"T0000000083","MV":"Tritium Pty Ltd","MS":"123","MF":"1.009","IS":"false","IL":"HEARSAY","IF":["RFID_NONE",
"OCPP_NONE"],"IT":"LOCAL","ID":"","VS":"E9YJGGBHZ1ME","RD":[{"TM":"2021-09-15T07:24:34,000+0200 S","TX":"B","
RV":"0.000","RU":"kWh","ST":"G","RI":"1-b:1.8.e","UT":" "},{"TM":"2021-09-15T07:25:18,000+0200 S","TX":"C","
RV":"3.960","RU":"kWh","ST":"G","RI":"1-b:1.8.e","UT":" "},{"TM":"2021-09-15T07:25:18,000+0200 S","TX":"E","
RV":"3.960","RU":"kWh","ST":"G","RI":"1-b:1.8.e","UT":" "}]}|{"SD":"
304402201B15EE6420899D06A3C70880DDCB580BDB2E0F06EADF25719D7515C61D911785022004D42601796C2ADC9853D3AF82F3C052FCBE
B4F0BC20485D29FD3F2C73D3E632"}</signedData>
                <publicKey encoding="plain"
>3059301306072A8648CE3D020106082A8648CE3D030107034200044 7AFFAB654C32B8A802689456050E4FF36D5CE23960F7E20A0825D7DB
7F65B1EA09086ADB91047B891420B2D301E7441C141CFAEECE2A3E09DDBFF49F883AC37</publicKey>
        </value>
</values>
```

Energy Delivery Session Identifier (EDSI)

The EDSI is used to:

1. Identify a charge session to the customer.
2. Avoid 'authentication of the user' requirements on the DC Meter.

Note 1 of AUTHZ002 recommends "a randomly generated transaction ID". However, it doesn't recommend the length of the ID.

To comply with the OCMF's pagination field, the identifier should be non-random, incrementing after every session. 32 bit length ID.

Tritium will ensure that each delivery of energy carries an identifier for the legally relevant receipt of energy delivery through the creation of a "Energy Delivery Session Identifier" (EDSI).

1. The EDSI shall be created on the DC Meter in response to the start of an energy delivery session.
2. The EDSI shall be an input to a hashing function, the result of which will be shown on the legally relevant portion of the display.
3. The EDSI shall be stored with the legally relevant measured values on the DC Meter.
4. The EDSI shall be included in the legally relevant "Digital Receipt" that is transmitted to the Charge Point Operator via OCPP and forms part of the cryptographically verifiable legally relevant data receipt (SAFE Initiative).
5. When an authentication system is in use, which is covered by one or more standards and certifications suitable for use with EVSE, and a certified identity system can provide a verifiable ID for the authenticated Loader User, that ID can be included in the DC Meter's Legally relevant display, storage, and digital receipt.
6. The EDSI and the authentication ID (when available) shall be stored on the DC Meter together with the legally relevant measured energy delivered.

Session Signature

The session signature is a 12-character, 5-bit encoded string that allows a charge session to be uniquely identified.

The character set for the string is as follows:

```
0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F,G,H,J,K,L,M,N,P,R,S,T,U,V,X,Y,Z
```

The Session Signature is displayed on the legally relevant display, and also appears in the "VS" field of the SAFE message/OCMF receipt. The session signature can hence be used by an end user to search the CPO's receipt database with a unique string, and retrieve the OCMF receipt that matches their charging session.

Networking Interfaces

There are several interfaces with the DC Meter to a number of different actors/purposes. There are differing security requirements for the different interfaces. This document gives an overview of those different interfaces, their purpose and their security.

Interface	Purpose	Transport Layer	Security Mechanism	Trust Root/Private Key Management	Notes
Trusted Technician Interface	• Telemetry • Calibration • Commissioning • Configuration • Testing • Servicing	TLS 1.2	TLS connection. x.509 certificates with adjustable permissions	TTI Certificate Authority - Tritium controlled	See Trusted Technician Interface (TTI) Certificate for read-only can be made readily available. Server (DCM) authentication is not critical; only client (PC) authentication for inputs.
Trusted Screen Overlay*	Display of information to user	TCP echoed onto SPI	Information exchanged is signed by private keys. Keys are signed by Trust Anchor to authenticate.	TSO and DCM generate unique keys onboard	See Pairing TSO and DCM
Trust Anchor	Authentication of Tritium product	TCP	Physical location security, not networked (air gapped)	Trust Anchor physically secured - Tritium controlled	See Trust Anchor Communication
SAFE*	Signature of billing receipt	(via Charger Communications)	Information is signed by DCM private key.	DCM generates unique key onboard	See Supporting the SAFE Initiative
CPO Communication	• (Optional) Change tariff	(via Charger Communications)	Information is signed by CPO's private key	CPO public key written to DCM EEPROM during configuration	See Charger to DC Meter Communications
Charger Communications (a.k.a MQTT)	• Communication with charger • Forwards communication with CPO	MQTT via TLS 1.2	Security mechanism is not critical as legally-relevant information is signed	Charger has certificate generated by MQTT Certificate Authority - Tritium controlled	See Charger to DC Meter Communications
Debug print out	Debug messages out	UDP	N/A	N/A	Not legally-relevant data. No input
Time	Time synchronisation	UDP	N/A	N/A	NTP
Update Server	Hosting firmware image for bootloading	TLS 1.2	TLS connection.	Image Hosting Certificate Authority - Tritium controlled	See Application Package Manager
Update Image	Signing of firmware image	(via Update Server)	Binary image is signed by certificate authority's private key.	Image Signing Certificate Authority - Tritium controlled	See Application Package Manager

*DC Meter uses same private/public key for TSO and SAFE signing.

Certificate Authority

All Tritium controlled certificate authorities are kept on a secure GIT repository (separate from main source repository), only accessible by minimal personnel. The root keys are kept there and only leaf certificates are issued to the appropriate receiver e.g. TTI production access certificate generated and then given to secure room production personnel.

Charger to DC Meter Communications

The charger and DC meter communicate via a MQTT broker.

MQTT Topics

List of what topics the DC meter system should subscribe and publish to, over MQTT, as a tree structure is listed below:

Publish

- DCMS
  - Telemetry
    - Status
      - Heartbeat
      - State
      - Error Info
      - Secondary Info
      - OoS
      - Stop Charge
    - Voltage
    - Current
    - Power
    - Temperature
      - Board
      - Shunt
      - AFE
  - Info
    - TME (Total Metered Energy)
    - Firmware
    - Public Key
    - Serial ID
  - Session
    - EDSI*
    - Energy
    - Duration
  - CSMS
    - SAFE** ('JSON_base64': SessionID + EDSI + SAFE)
    - CPO Public Key
    - Active Tariff
    - Queued Tariff

Subscribe

- DCMS
  - Charger
    - Session*
      - ID (hexadecimal string, fixed-length 16 bytes, example: "74EF112C541A8741")
      - Signature
  - CSMS
    - Tarrifs**

*Session ID and EDSI are paired so Session ID should be pushed by MQTT broker when EDSI is received.

**File containing data and signature

Cryptographic Security

The mbedTLS library includes an entropy accumulator module that handles many requirements for random bit generation. Multiple inputs are SHA256 together. Sources of entropy can be seen in Entropy source.

All critical transmitted data is signed by the meter with a ECDSA with 256 bit key length.

All keys and signatures will be derived from secp256r1 curve (256-bit).

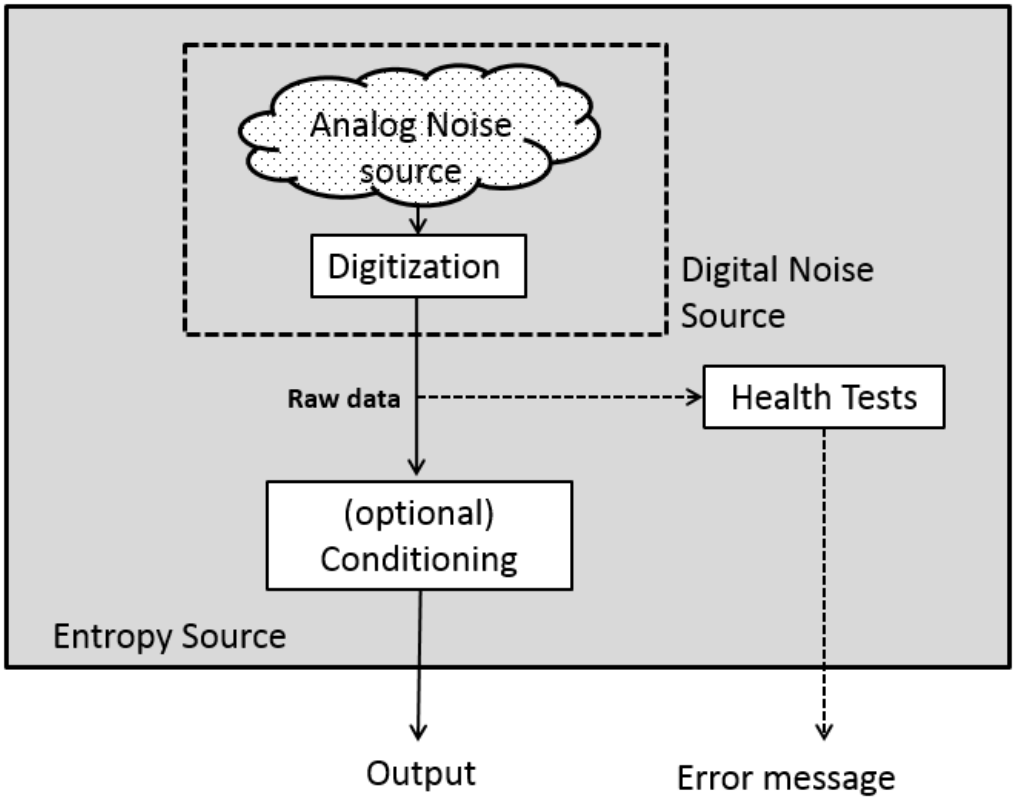
All hashes calculated on the meter are SHA256.

Private keys for TSO and SAFE signing are never displayed, exported, or read from the DC Meter at any time. A new private key may be generated, but never seen. A new private key is only generated in production prior to being signed by the Trust Anchor.

Entropy source

Model

Entropy source used in DCM project was created and evaluated according to recommendations in NIST SP 800-90B with below model:



Analogue noise source is composed of some set of the below:

- Internal processor temperature measurement (TSO)
- Voltage on a floating pin (TSO)
- Zener diode reverse voltage (DCM)
- System supply voltage (DCM)
- Board temperature measurement (DCM)

Analogue voltages from above sources are read by ADC. Only two least significant bits are used. Four independent inputs are sampled and ORed to create 8-bit sample of entropy which is transferred to entropy pool. No conditioning is performed.

Health Tests

Entropy source implements below health tests:

- Repetition Count Test
- Adaptive Proportion Test

These tests are run continuously during entropy source operation analysing every generated entropy sample. Startup self-test runs continuous tests on 1024 consecutive samples before allowing any reads from the entropy source (1024 entropy samples are discarded).

Cutoff values for health tests was calculated assuming the entropy would be at most 4 bits per 1 byte and false positive probability of $= 2^{20}$. For lower entropy estimates those values are more conservative and allow for faster detection of any problems with entropy source.

Entropy Estimate TSO

SP800-90B_EntropyAssessment test suit was used to perform estimation of the entropy source.

Initial entropy estimate results:

Tested data: TSO_ENTROPY 

Loaded 125000 samples of 256 distinct 8-bit-wide symbols
Number of Binary Symbols: 1000000

***** Warning: data contains less than 1000000 samples *****

Running non-IID tests...

Running Most Common Value Estimate...
Bitstring MCV Estimate: mode = 500588, p-hat = 0.50058800000000003, p_u = 0.501875914405154
Most Common Value Estimate (bit string) = 0.994597 / 1 bit(s)
Literal MCV Estimate: mode = 3008, p-hat = 0.024063999999999999, p_u = 0.025180499129802431
Most Common Value Estimate = 5.311549 / 8 bit(s)

Running Entropic Statistic Estimates (bit strings only)...
Bitstring Collision Estimate: X-bar = 2.4761705488932746, sigma-hat = 0.49943245278882376, p = 0.61369651531708058
Collision Test Estimate (bit string) = 0.704403 / 1 bit(s)
Bitstring Markov Estimate: P_0 = 0.49941200000000002, P_1 = 0.50058800000000003, P_0,0 = 0.51553930530164538, P_0,1 = 0.48446069469835462, P_1,0 = 0.48332161378219213, P_1,1 = 0.51667838621780793, p_max = 1.89870646284791 54e-37
Markov Test Estimate (bit string) = 0.953018 / 1 bit(s)
Bitstring Compression Estimate: X-bar = 5.095580078444673, sigma-hat = 1.0453171279805322, p = 0.095732601400006612
Compression Test Estimate (bit string) = 0.564141 / 1 bit(s)

Running Tuple Estimates...
Bitstring t-Tuple Estimate: t = 26, p-hat_max = 0.67867812579184883, p_u = 0.67988099766088372
Bitstring LRS Estimate: u = 27, v = 50, p-hat = 0.59412595716394034, p_u = 0.5953908455323621
T-Tuple Test Estimate (bit string) = 0.556646 / 1 bit(s)
Literal t-Tuple Estimate: t = 3, p-hat_max = 0.076970433136702238, p_u = 0.078912363185281567
Literal LRS Estimate: u = 4, v = 6, p-hat = 0.025198790585647265, p_u = 0.026340647506713166
T-Tuple Test Estimate = 3.663605 / 8 bit(s)
LRS Test Estimate (bit string) = 0.748091 / 1 bit(s)
LRS Test Estimate = 5.246565 / 8 bit(s)

Running Predictor Estimates...
Bitstring MultiMCW Prediction Estimate: N = 999937, Pglobal' = 0.5099518083247998 (C = 508632) Plocal can't affect result (r = 23)
Multi Most Common in Window (MultiMCW) Prediction Test Estimate (bit string) = 0.971567 / 1 bit(s)
Literal MultiMCW Prediction Estimate: N = 124937, Pglobal' = 0.026690541968189662 (C = 3191) Plocal = 0.066460295690674007 (r = 6)
Multi Most Common in Window (MultiMCW) Prediction Test Estimate = 3.911363 / 8 bit(s)
Bitstring Lag Prediction Estimate: N = 999999, Pglobal' = 0.62768868032127301 (C = 626442) Plocal can't affect result (r = 36)
Lag Prediction Test Estimate (bit string) = 0.671879 / 1 bit(s)
Literal Lag Prediction Estimate: N = 124999, Pglobal' = 0.024517925509921403 (C = 2927) Plocal = 0.038411357486915501 (r = 5)
Lag Prediction Test Estimate = 4.702323 / 8 bit(s)
Bitstring MultiMMC Prediction Estimate: N = 999998, Pglobal' = 0.63590459527860699 (C = 634663) Plocal = 0.67503520854902899 (r = 44)
Multi Markov Model with Counting (MultiMMC) Prediction Test Estimate (bit string) = 0.566965 / 1 bit(s)
Literal MultiMMC Prediction Estimate: N = 124998, Pglobal' = 0.046616736890156975 (C = 5638) Plocal = 0.066454823913203537 (r = 6)
Multi Markov Model with Counting (MultiMMC) Prediction Test Estimate = 3.911482 / 8 bit(s)
Bitstring LZ78Y Prediction Estimate: N = 999983, Pglobal' = 0.51735603414128084 (C = 516060) Plocal can't affect result (r = 22)
LZ78Y Prediction Test Estimate (bit string) = 0.950771 / 1 bit(s)
Literal LZ78Y Prediction Estimate: N = 124983, Pglobal' = 0.04666297151055105 (C = 5643) Plocal = 0.066456169129722908 (r = 6)
LZ78Y Prediction Test Estimate = 3.911453 / 8 bit(s)

H_original: 3.663605
H_bitstring: 0.556646

min(H_original, 8 X H_bitstring): 3.663605

Restart test result:



Tested data: TSO_ENTROPY_RESTART

```
Loaded 125000 samples of 256 distinct 8-bit-wide symbols
Number of Binary Symbols: 1000000

*** Warning: data contains less than 1000000 samples ***


Running non-IID tests...

Running Most Common Value Estimate...
Bitstring MCV Estimate: mode = 504230, p-hat = 0.50422999999999996, p_u = 0.50551786920582831
Most Common Value Estimate (bit string) = 0.984166 / 1 bit(s)
Literal MCV Estimate: mode = 2932, p-hat = 0.023456000000000001, p_u = 0.024558647493909624
Most Common Value Estimate = 5.347625 / 8 bit(s)

Running Entropic Statistic Estimates (bit strings only)...
Bitstring Collision Estimate: X-bar = 2.467260619727762, sigma-hat = 0.4989275971047431, p = 0.63182950613234823
Collision Test Estimate (bit string) = 0.662393 / 1 bit(s)
Bitstring Markov Estimate: P_0 = 0.49576999999999999, P_1 = 0.50422999999999996, P_0,0 = 0.51573212524381318, P_0,1 =
0.48426787475618682, P_1,0 = 0.47614184003331811, P_1,1 = 0.52385815996668184, p_max = 1.10
34947029349809e-36
Markov Test Estimate (bit string) = 0.933182 / 1 bit(s)
Bitstring Compression Estimate: X-bar = 5.0538494665350857, sigma-hat = 1.0781510509989141, p = 0.11185973493558654
Compression Test Estimate (bit string) = 0.526706 / 1 bit(s)

Running Tuple Estimates...
Bitstring t-Tuple Estimate: t = 30, p-hat_max = 0.7135138695884492, p_u = 0.71467845221570114
Bitstring LRS Estimate: u = 31, v = 51, p-hat = 0.59643604941784989, p_u = 0.59769978269478419
T-Tuple Test Estimate (bit string) = 0.484634 / 1 bit(s)
Literal t-Tuple Estimate: t = 3, p-hat_max = 0.070607343281027177, p_u = 0.072473672966833283
Literal LRS Estimate: u = 4, v = 6, p-hat = 0.022449570231439998, p_u = 0.023528858388111841
T-Tuple Test Estimate = 3.786399 / 8 bit(s)
LRS Test Estimate (bit string) = 0.742507 / 1 bit(s)
LRS Test Estimate = 5.409425 / 8 bit(s)

Running Predictor Estimates...
Bitstring MultiMCW Prediction Estimate: N = 999937, Pglobal' = 0.52001373198811962 (C = 518694) Plocal can't affect result (r = 18)
Multi Most Common in Window (MultiMCW) Prediction Test Estimate (bit string) = 0.943378 / 1 bit(s)
Literal MultiMCW Prediction Estimate: N = 124937, Pglobal' = 0.024415453299536687 (C = 2913) Plocal = 0.066460295709743725 (r = 6)
Multi Most Common in Window (MultiMCW) Prediction Test Estimate = 3.911363 / 8 bit(s)
Bitstring Lag Prediction Estimate: N = 999999, Pglobal' = 0.61445308847673119 (C = 613198) Plocal = 0.64771283626842591 (r = 40)
Lag Prediction Test Estimate (bit string) = 0.626574 / 1 bit(s)
Literal Lag Prediction Estimate: N = 124999, Pglobal' = 0.023781249039914971 (C = 2837) Plocal = 0.038411357486628259 (r = 5)
Lag Prediction Test Estimate = 4.702323 / 8 bit(s)
Bitstring MultiMMC Prediction Estimate: N = 999998, Pglobal' = 0.62500508953117651 (C = 623756) Plocal = 0.68724003211083529 (r = 46)
Multi Markov Model with Counting (MultiMMC) Prediction Test Estimate (bit string) = 0.541114 / 1 bit(s)
Literal MultiMMC Prediction Estimate: N = 124998, Pglobal' = 0.041086572026887987 (C = 4958) Plocal = 0.066454823919652975 (r = 6)
Multi Markov Model with Counting (MultiMMC) Prediction Test Estimate = 3.911482 / 8 bit(s)
Bitstring LZ78Y Prediction Estimate: N = 999983, Pglobal' = 0.52109775168740846 (C = 519802) Plocal can't affect result (r = 18)
LZ78Y Prediction Test Estimate (bit string) = 0.940374 / 1 bit(s)
Literal LZ78Y Prediction Estimate: N = 124983, Pglobal' = 0.041058945120268975 (C = 4954) Plocal = 0.066456169140905311 (r = 6)
LZ78Y Prediction Test Estimate = 3.911453 / 8 bit(s)

H_original: 3.786399
H_bitstring: 0.484634

min(H_original, 8 X H_bitstring): 3.786399
```

Conclusion

Lowest entropy estimate from performed tests was 3.66 bits per 1 byte. To avoid security risks entropy was assumed to be 2 bits per 1 byte. That required the seed for Random Number Generator to be 128-bytes long to meet the requirement of security_strength = 256 (according to NIST 800-90A).

Entropy Estimate DCMeter

SP800-90B_EntropyAssessment test suit was used to perform estimation of the entropy source.

Initial entropy estimate results:

Tested data: DCM_ENTROPY



Loaded 125000 samples of 256 distinct 8-bit-wide symbols
Number of Binary Symbols: 1000000

***** Warning: data contains less than 1000000 samples *****

Running non-IID tests...

Running Most Common Value Estimate...
Bitstring MCV Estimate: mode = 512150, p-hat = 0.51214999999999999, p_u = 0.51343753498903066
Most Common Value Estimate (bit string) = 0.961739 / 1 bit(s)
Literal MCV Estimate: mode = 697, p-hat = 0.0055760000000000002, p_u = 0.0061185140517987608
Most Common Value Estimate = 7.352603 / 8 bit(s)

Running Entropic Statistic Estimates (bit strings only)...
Bitstring Collision Estimate: X-bar = 2.5014445842474027, sigma-hat = 0.499998538531707, p = 0.51721002500903523
Collision Test Estimate (bit string) = 0.951178 / 1 bit(s)
Bitstring Markov Estimate: P_0 = 0.48785000000000001, P_1 = 0.51214999999999999, P_0,0 = 0.48643640463257148, P_0,1 = 0.51356359536742846, P_1,0 = 0.48919552708293873, P_1,1 = 0.51080447291706133, p_max = 4.54
71889905616428e-38
Markov Test Estimate (bit string) = 0.969127 / 1 bit(s)
Bitstring Compression Estimate: X-bar = 5.2170587287665198, sigma-hat = 1.0146958526204284, p = 0.029834688671164455
Compression Test Estimate (bit string) = 0.844478 / 1 bit(s)

Running Tuple Estimates...
Bitstring t-Tuple Estimate: t = 16, p-hat_max = 0.52662886552610955, p_u = 0.52791495301242475
Bitstring LRS Estimate: u = 17, v = 37, p-hat = 0.50098816908640509, p_u = 0.50227608186689265
T-Tuple Test Estimate (bit string) = 0.921623 / 1 bit(s)
Literal t-Tuple Estimate: t = 1, p-hat_max = 0.0055760000000000002, p_u = 0.0061185140517987608
Literal LRS Estimate: u = 2, v = 4, p-hat = 0.0047568950570268933, p_u = 0.0052581863764200347
T-Tuple Test Estimate = 7.352603 / 8 bit(s)
LRS Test Estimate (bit string) = 0.993448 / 1 bit(s)
LRS Test Estimate = 7.571219 / 8 bit(s)

Running Predictor Estimates...
Bitstring MultiMCW Prediction Estimate: N = 999937, Pglobal' = 0.50970480363292847 (C = 508385) Plocal can't affect result (r = 19)
Multi Most Common in Window (MultiMCW) Prediction Test Estimate (bit string) = 0.972266 / 1 bit(s)
Literal MultiMCW Prediction Estimate: N = 124937, Pglobal' = 0.0052219840364005248 (C = 590) Plocal can't affect result (r = 3)
Multi Most Common in Window (MultiMCW) Prediction Test Estimate = 7.581186 / 8 bit(s)
Bitstring Lag Prediction Estimate: N = 999999, Pglobal' = 0.51598887379460168 (C = 514701) Plocal can't affect result (r = 23)
Lag Prediction Test Estimate (bit string) = 0.954588 / 1 bit(s)
Literal Lag Prediction Estimate: N = 124999, Pglobal' = 0.0055895910672354868 (C = 634) Plocal can't affect result (r = 3)
Lag Prediction Test Estimate = 7.483042 / 8 bit(s)
Bitstring MultiMMC Prediction Estimate: N = 999998, Pglobal' = 0.51340756239112006 (C = 512119) Plocal can't affect result (r = 20)
Multi Markov Model with Counting (MultiMMC) Prediction Test Estimate (bit string) = 0.961824 / 1 bit(s)
Literal MultiMMC Prediction Estimate: N = 124998, Pglobal' = 0.0050425074976730745 (C = 569) Plocal can't affect result (r = 3)
Multi Markov Model with Counting (MultiMMC) Prediction Test Estimate = 7.631643 / 8 bit(s)
Bitstring LZ78Y Prediction Estimate: N = 999983, Pglobal' = 0.51341425354430159 (C = 512118) Plocal can't affect result (r = 20)
LZ78Y Prediction Test Estimate (bit string) = 0.961805 / 1 bit(s)
Literal LZ78Y Prediction Estimate: N = 124983, Pglobal' = 0.0050515424860314824 (C = 570) Plocal can't affect result (r = 3)
LZ78Y Prediction Test Estimate = 7.629060 / 8 bit(s)

H_original: 7.352603
H_bitstring: 0.844478

min(H_original, 8 X H_bitstring): 6.755821

Restart test result:



Tested data: DCM_ENTROPY_RESTART

Loaded 125000 samples of 256 distinct 8-bit-wide symbols
Number of Binary Symbols: 1000000

***** Warning: data contains less than 1000000 samples *****

Running non-IID tests...

Running Most Common Value Estimate...
Bitstring MCV Estimate: mode = 512090, p-hat = 0.51209000000000005, p_u = 0.51337753873641978
Most Common Value Estimate (bit string) = 0.961908 / 1 bit(s)
Literal MCV Estimate: mode = 692, p-hat = 0.0055360000000000001, p_u = 0.0060765755315594508
Most Common Value Estimate = 7.362526 / 8 bit(s)

Running Entropic Statistic Estimates (bit strings only)...
Bitstring Collision Estimate: X-bar = 2.5025876782469858, sigma-hat = 0.49999392951679228, p = 0.5
Collision Test Estimate (bit string) = 1.000000 / 1 bit(s)
Bitstring Markov Estimate: P_0 = 0.48791000000000001, P_1 = 0.51208999999999993, P_0,0 = 0.48456477629070938, P_0,1 = 0.51543522370929062, P_1,0 = 0.49109822706599832, P_1,1 = 0.50890177293400174, p_max = 2.8304775140527792e-38
Markov Test Estimate (bit string) = 0.974470 / 1 bit(s)
Bitstring Compression Estimate: X-bar = 5.2119763366628451, sigma-hat = 1.0181239430763833, p = 0.034900323605197014
Compression Test Estimate (bit string) = 0.806769 / 1 bit(s)

Running Tuple Estimates...
Bitstring t-Tuple Estimate: t = 16, p-hat_max = 0.52934264357354366, p_u = 0.53062833919023844
Bitstring LRS Estimate: u = 17, v = 41, p-hat = 0.51886258802425067, p_u = 0.52014958652029042
T-Tuple Test Estimate (bit string) = 0.914226 / 1 bit(s)
Literal t-Tuple Estimate: t = 1, p-hat_max = 0.0055360000000000001, p_u = 0.0060765755315594508
Literal LRS Estimate: u = 2, v = 4, p-hat = 0.0044267896543079858, p_u = 0.0049104548693724498
T-Tuple Test Estimate = 7.362526 / 8 bit(s)
LRS Test Estimate (bit string) = 0.943002 / 1 bit(s)
LRS Test Estimate = 7.669928 / 8 bit(s)

Running Predictor Estimates...
Bitstring MultiMCW Prediction Estimate: N = 999937, Pglobal' = 0.50962980214602827 (C = 508310) Plocal can't affect result (r = 23)
Multi Most Common in Window (MultiMCW) Prediction Test Estimate (bit string) = 0.972478 / 1 bit(s)
Literal MultiMCW Prediction Estimate: N = 124937, Pglobal' = 0.0054830023635195945 (C = 621) Plocal can't affect result (r = 3)
Multi Most Common in Window (MultiMCW) Prediction Test Estimate = 7.510818 / 8 bit(s)
Bitstring Lag Prediction Estimate: N = 999999, Pglobal' = 0.5051203819376 (C = 503832) Plocal can't affect result (r = 21)
Lag Prediction Test Estimate (bit string) = 0.985301 / 1 bit(s)
Literal Lag Prediction Estimate: N = 124999, Pglobal' = 0.0047218776996898939 (C = 531) Plocal can't affect result (r = 3)
Lag Prediction Test Estimate = 7.726424 / 8 bit(s)
Bitstring MultiMMC Prediction Estimate: N = 999998, Pglobal' = 0.51314257823112464 (C = 511854) Plocal can't affect result (r = 23)
Multi Markov Model with Counting (MultiMMC) Prediction Test Estimate (bit string) = 0.962568 / 1 bit(s)
Literal MultiMMC Prediction Estimate: N = 124998, Pglobal' = 0.0051436296964943876 (C = 581) Plocal can't affect result (r = 3)
Multi Markov Model with Counting (MultiMMC) Prediction Test Estimate = 7.602998 / 8 bit(s)
Bitstring LZ78Y Prediction Estimate: N = 999983, Pglobal' = 0.51328825923540855 (C = 511992) Plocal can't affect result (r = 23)
LZ78Y Prediction Test Estimate (bit string) = 0.962159 / 1 bit(s)
Literal LZ78Y Prediction Estimate: N = 124983, Pglobal' = 0.0051610973672557353 (C = 583) Plocal can't affect result (r = 3)
LZ78Y Prediction Test Estimate = 7.598106 / 8 bit(s)

H_original: 7.362526
H_bitstring: 0.806769

min(H_original, 8 X H_bitstring): 6.454154

Conclusion

Lowest entropy estimate from performed tests was 6.45 bits per 1 byte. To avoid security risks and having consistency in the project entropy was assumed to be 2 bits per 1 byte. That required the seed for Random Number Generator to be 128-bytes long to meet the requirement of security_strength = 256 (according to NIST 800-90A).

Random numbers

Random numbers generator was tested according to NIST SP 800-22 . (Bitstream count: 100, bitstream length 100000)

Random Numbers test TSO

Tested data: TSO_RNG_10M

All statistical tests in sts-2.1.2 test suite have passed. Full report: finalAnalysisReport.txt

Random Numbers test DCM

Tested data: DCM_RNG_10M

All statistical tests in sts-2.1.2 test suite have passed. Full report: finalAnalysisReport.txt

External Non Volatile Storage

Overview

The microcontroller on the DC Meter does not have enough internal non-volatile storage to satisfy all its purposes including a long-term logbook-based auditing system. The DC Meter has been designed to have service period of 10 years. During the service period, every event and configuration change needs to be logged.

The external non volatile storage is located on the controller board of the meter, protected by tamper evident seal.

The external non volatile storage is used for a number purposes:

- Audit Event Log - saves audit events over the entire life of the meter.
  - See Audit Event Log
  - More than 600 blocks or 150 MB are reserved for the Audit Event Log
- Charge Session Log - saves information to reproduce SAFE receipt
  - See Charge Session Log
  - More than 300 blocks or 75 MB are reserved for the Charge Session Log
- Application Package Manager - saves the downloaded firmware package of the meter to allow boot loading
  - See Application Package Manager
  - 4 blocks or 1 MB are reserved for the Application Package Manager
- Run Time Session Info - saves the auxiliary information needed to be kept in storage
  - 2 blocks or 512 kB are reserved for the Run Time Session Info

Run Time Session Info

The last EDSI used and Total Metered Energy are saved to the external flash after every charge session and read at startup. This allows these variables to be added upon throughout the whole life of the meter. These are saved with a hash to verify integrity.

Device Information and Write Methodology

Flash Memory Topology

Selecting the right flash-based storage device required a trade-off between cost, capacity and robustness. A planar Single-level Cell (SLC) topology NAND with Error Correcting Codes (ECC) was selected for its durability and longevity. NAND flash stores data in cells comprised of floating-gate transistors. The amount of charge stored in each cell determines the value of the data. If the charge in the cell become insufficient to determine the state, then the information will be lost. ECC technology can correct bits and restoring depleted erroneous cell charge.

Flash Memory Cell Endurance

Cell endurance is described as the number of program/erase (P/E) cycles a flash cell can undergo before data can no longer be sufficiently retained and the cell becomes unreliable. Every time a flash cell is written (one P/E cycle) the oxide layer of the cell is slightly degraded. As this layer weakens from the stress of P/E cycles it becomes easier for stored charge to leak, and the cell's ability to retain data is reduced.

Flash Data Retention

The level of charge in each cell must be kept within certain thresholds to maintain data integrity. Unfortunately, charge leaks from flash cells over time, and if too much charge is lost then the data stored will also be lost.

During normal operation, it is the flash driver's responsibility to routinely restore lost charge. However, when the flash is not powered the state of charge will naturally degrade with time. The rate of charge loss, and sensitivity of the flash to that loss, is impacted by the flash structure, amount of flash wear (number of P/E cycles performed on the cell), and the storage temperature.

Flash Memory Device Selection

Tritium selected the Toshiba 4Gb TC58CV SLC NAND with ECC device range for the DC Meter. Toshiba was contacted and provided internal reliability data.







Flash Memory Organisation

Feature	Size (in bytes)
Memory Capacity	(4096 + 128) x 64 pages x 2048 blocks
Memory Cell Buffer	4096 + 128
Page Size	4096 + 128
Block Size	256k + 8k

Flash Memory Performance

Symbol	Parameter	Min	Typ	Max	Unit
N_{VB}	Number of Valid Blocks	2008	-	2048	Block
t_{PROG}	Average Programming Time	-	450	600	us
t_{BERASE}	Block Erasing Time	-	2	7	ms
t_{R}	Cell Array to Buffer (with ECC)	-	115	280	us
N	Number of Partial Program Cycles in the Same Page	-	-	4	time

Flash Memory Limitations

1. NAND flash page cannot be individually erased.

- Tritium's strategy is to have a fixed length, page aligned, usage of blocks. Usage of memory is then easily detected between power cycles.
- File systems typically have some kind journaling system for meta-data to track file system changes. Changes to the file system, require changes in the journal. NAND Flash isn't well suited to journaling because the block sizes are large, and erasing is slow.
- Alternative: Third party options

Product	Comment
littlefs	https://github.com/ARMmbed/littlefs/blob/master/DESIGN.md "NAND flash already has many limitations that make it poorly suited for an embedded system: low erase cycles, very large blocks, errors that can develop even during reads, errors that can develop during writes of neighbouring blocks. Managing sensitive storage, such as NAND flash, is out of scope for the littlefs." Requires a Flash Translation Layer (FTL)
FreeRTOS+FAT	Requires a Flash Translation Layer (FTL)
yaffs	https://yaffs.net/ Yaffs supports a NAND flash. GPL-v2 Licence

2. Partial page programming.
   - ECC Parity Code generation restricts the number of writes (and where) to a page before invalidating the ECC system. Each page has a usable section of 4096 bytes. For each 512 bytes sector of a page, 16 bytes are reserved in an Out-of-band (OOB) area for ECC generation. The number of partial program cycles in the same page is 4, so the page is divided into 1024 bytes sectors.



Flash Memory Lifetime Concerns

The DC Meter could be running continuously at elevated temperature for its entire service life. The durability of the SLC NAND flash required internal quality testing.

Test	Iterations	Condition	Pass / Fail	Comment
Program/Erase each page (4096 bytes) in a block (64 pages) with alternating checker pattern data (0xAA/0x55)	100,000	25 °C	Pass	
Program/Erase each page (4096 bytes) in a block (64 pages) with alternating checker pattern data (0xAA/0x55)	100,000	80 °C	Fail	Some pages exhibited ECC uncorrectable after 45,000 iterations. Sufficient for our lifetime use.
Read disturb, continuous read of preprogrammed block, checking surrounding pages for corruption	100,000	25 °C	Pass	
Read disturb, continuous read of preprogrammed block, checking surrounding pages for corruption	100,000	80 °C	Pass	

Flash Memory Firmware: Page Organisation

- Records are fixed length (128 bytes) and include a SHA256 fingerprint for integrity checking.
- Records are to be sector aligned with eight records per sector.

Flash Memory Firmware: Power down

Several of the records kept are used in RAM and then flushed to disk. In the case of an unexpected power down (all power losses), the device has sufficient backup capacitance to detect the power drop, flush the records to flash before losing power.

Flash Memory Firmware: Bad Block Tracking

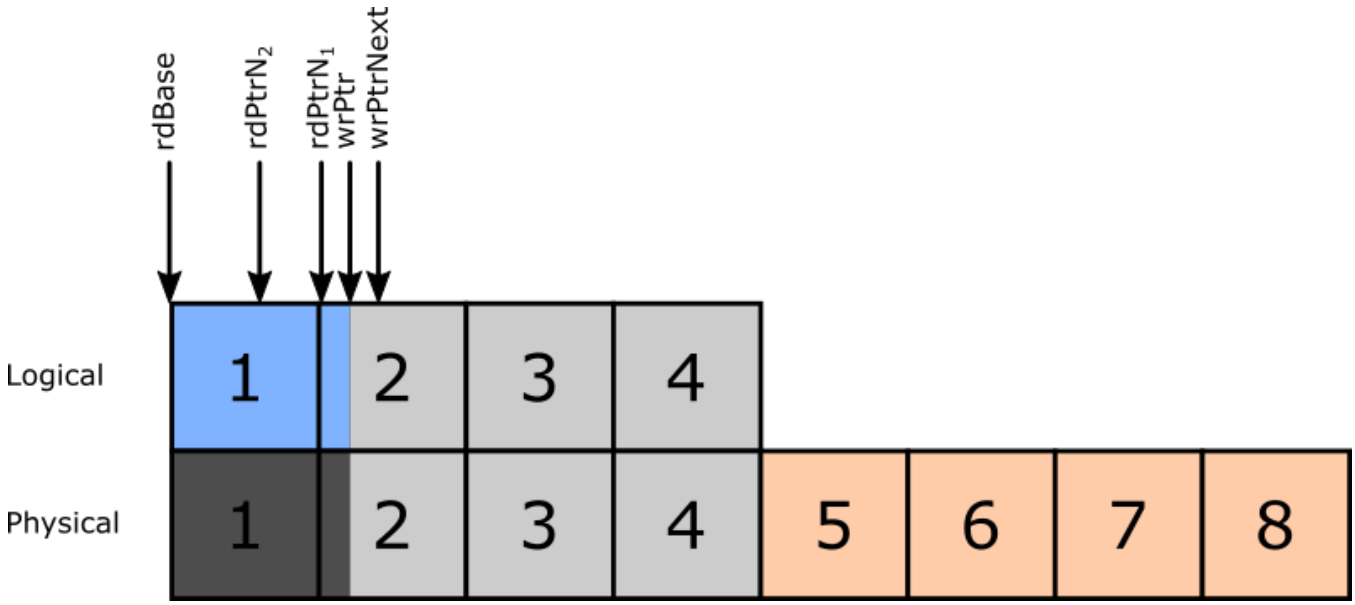
Block Allocation Table (BAT) section has been implemented to maintain a contiguous logical address space over a discontinuous physical address space.

- On first ever startup and when a new bad block is detected, the BAT is updated.
- There are 2048 blocks in a single plane of the TC58CV device.
    - 2048 bits (256 bytes) of storage is required to track bad blocks (1 bit / block).
    - Once a block has been marked bad, it will always be bad.

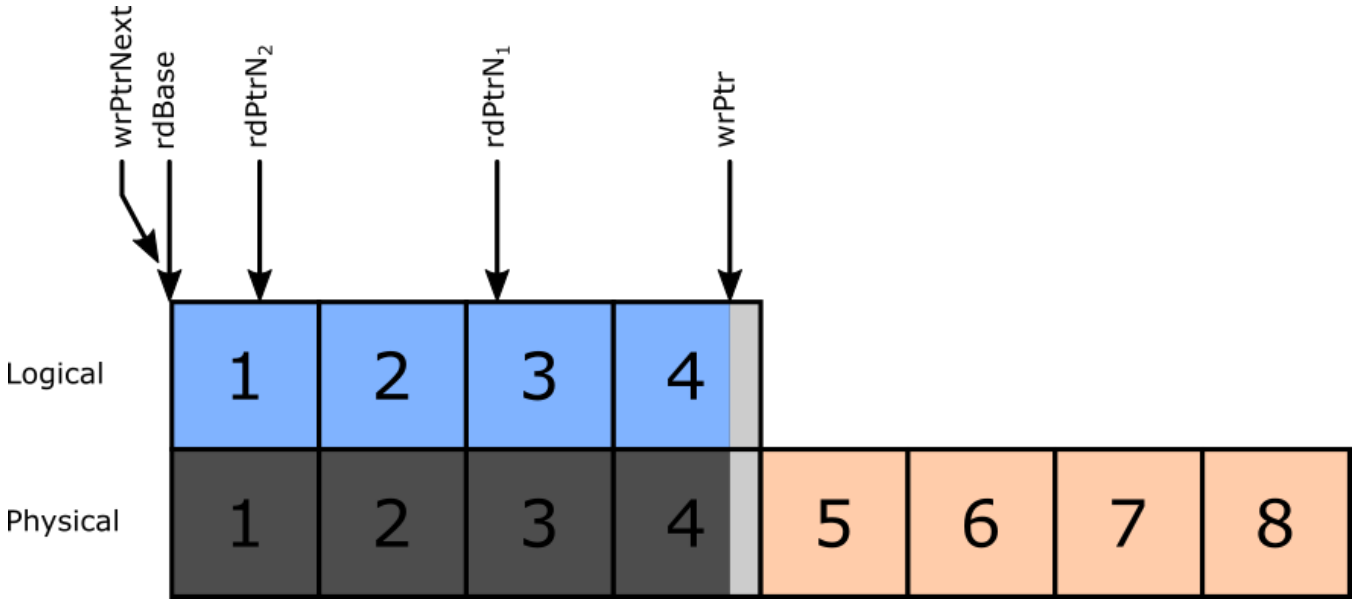


Flash Memory Firmware: Disk Manager

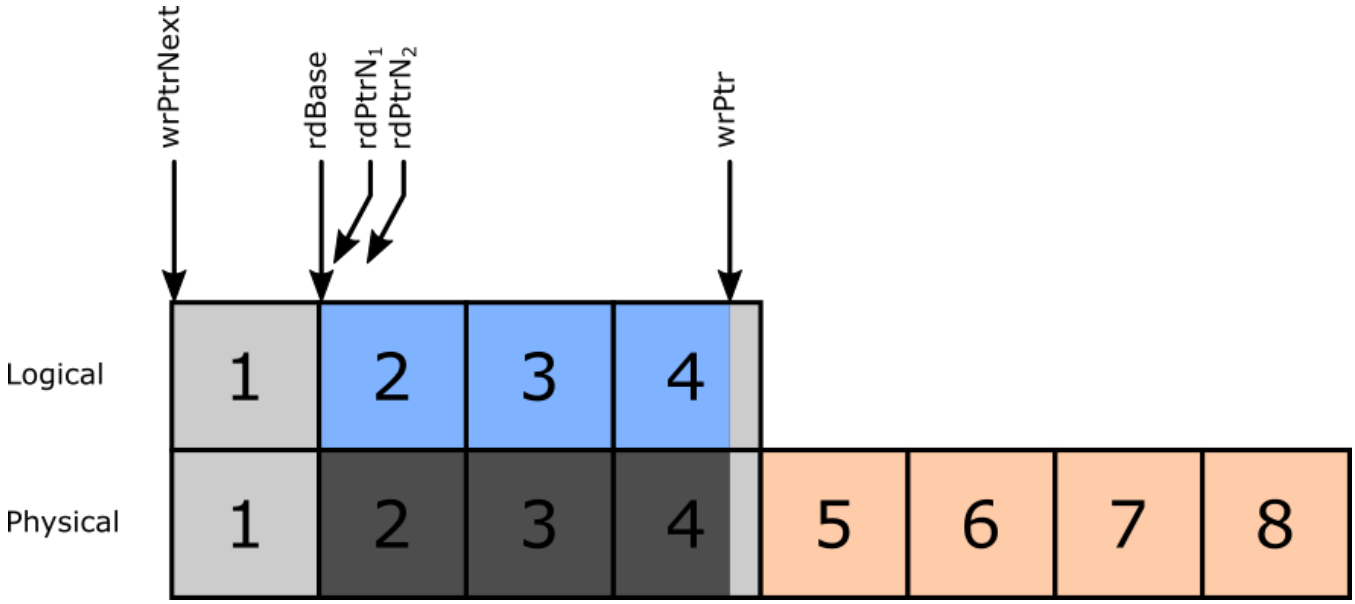
The Disk Manager is a component of the DC Meter which operates on the principal of a contiguous logical address space. Each file that is managed by the Disk Manager is allocated its address range in the form of a logically contiguous file partition. Each file partition contains operating policies (i.e. What to do when the partition is full, overwrite existing data or not).



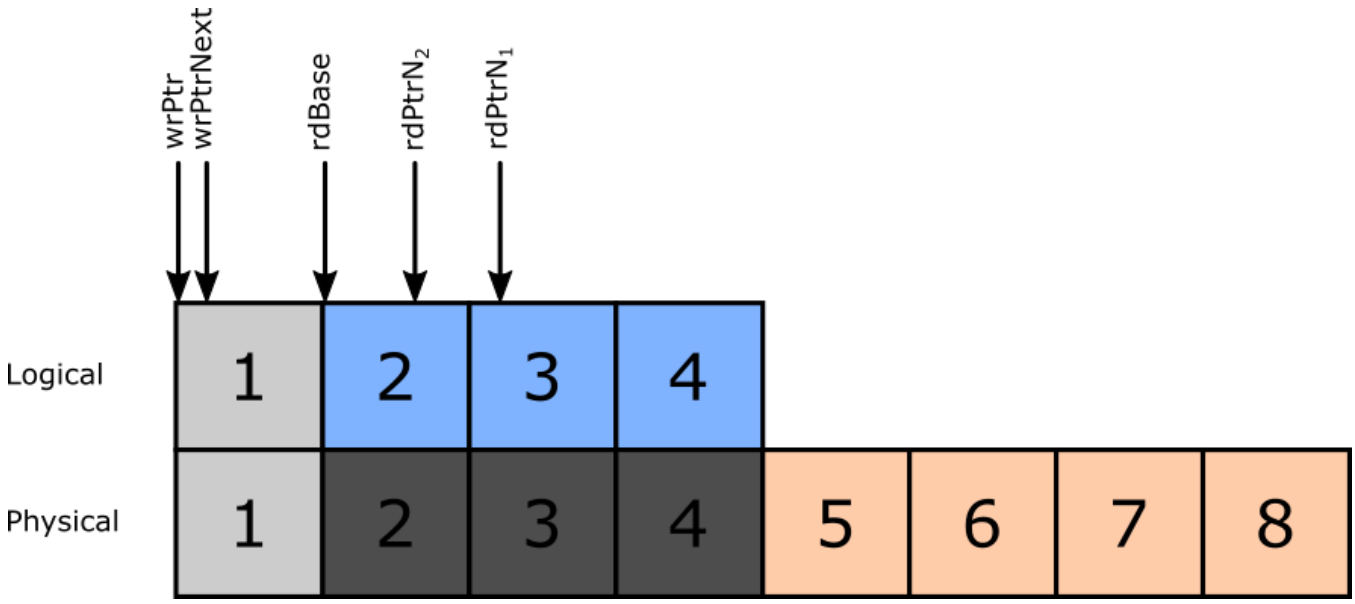
t1 - Normal operation, free space available in user partition.



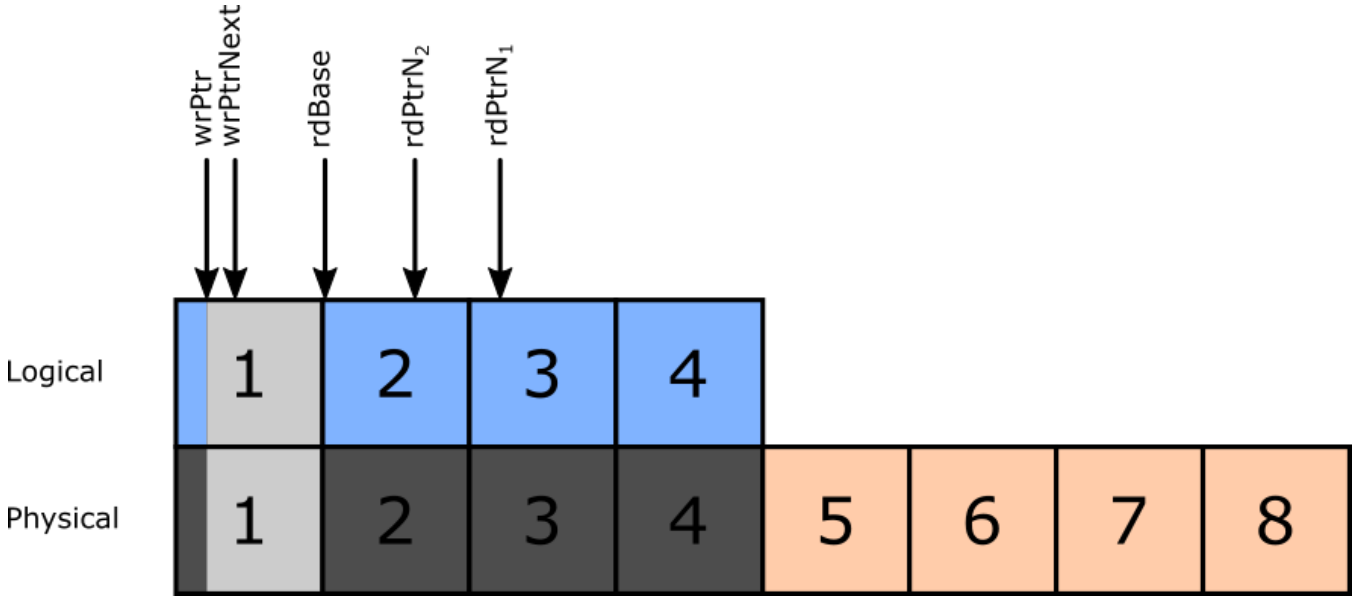
t2 - Normal operation, last page in user partition.



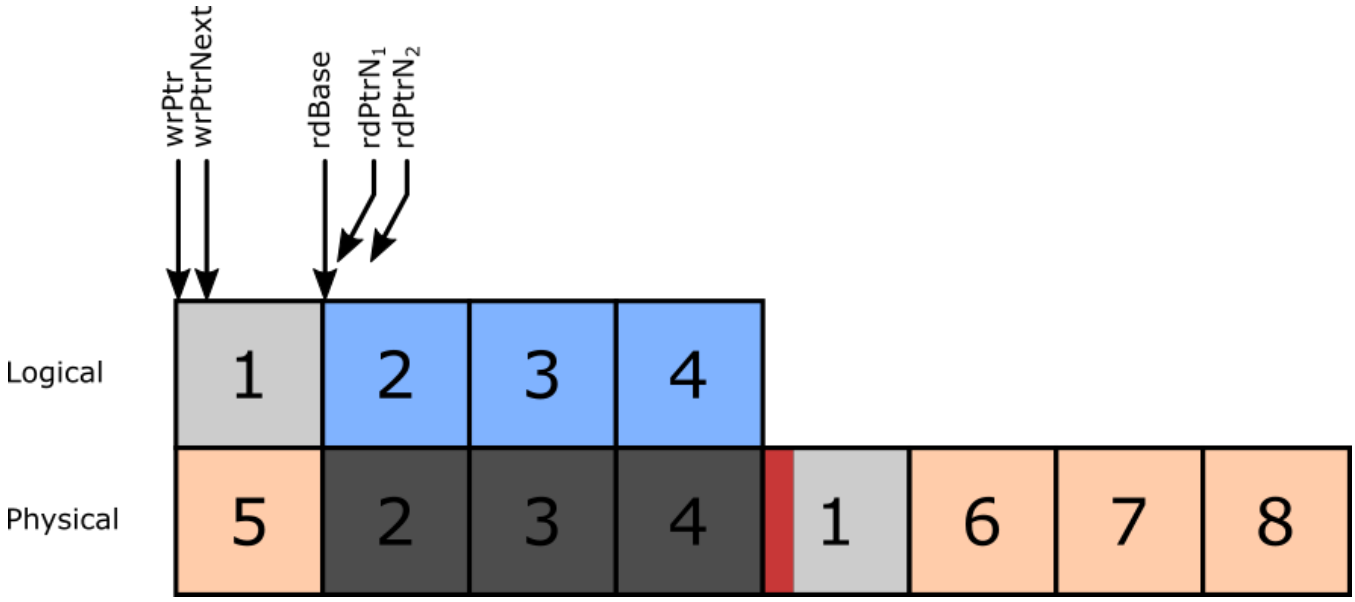
t3 - First block of the user partition is erased to make room for future data. The read base and iterator pointer are reset and relocated to the next logical block.



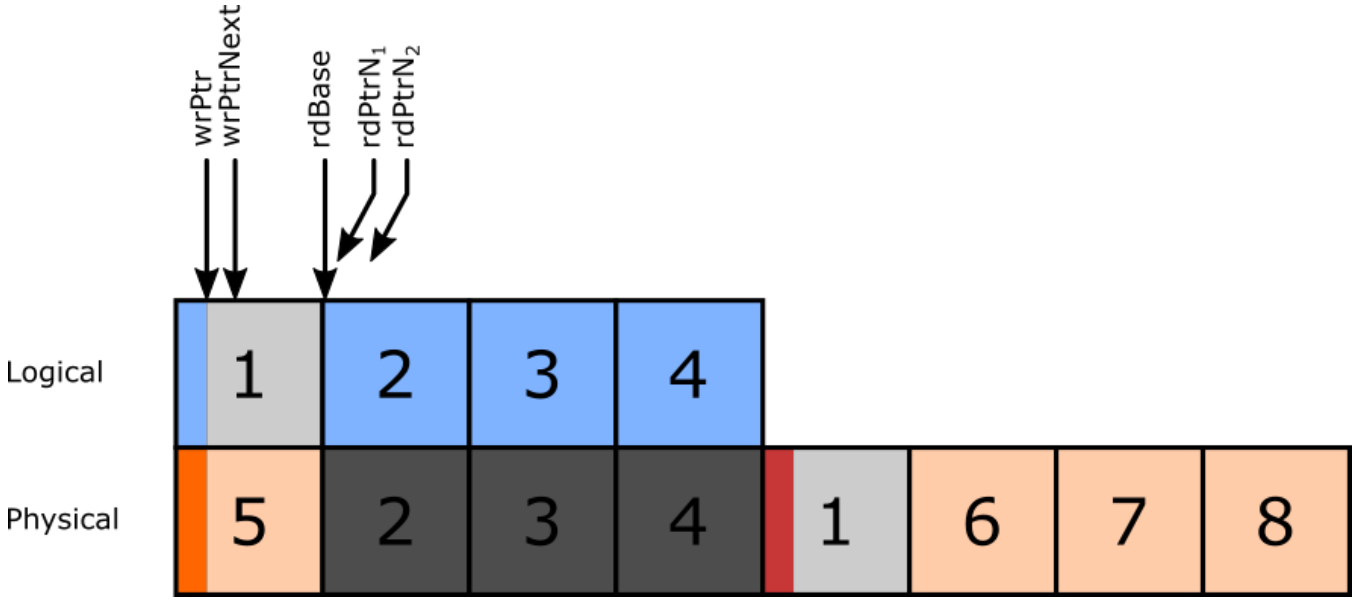
t4 - The user partition has wrapped. The write pointer is moved back to the first block of the user partition.



t5 - Normal operation, free space available in user partition.



t6 - A write operation failure occurs. Logical block 1 is reallocated. The write pointer is logically reset to start of the reallocated block. The contents of physical block 1 is orphaned (not deleted) during the reallocation process. The read iterators pointers are logically reset to protect iterators from pointing to potentially null data.



t7 - Normal operation, free space available in user partition.

Audit Event Log

Overview

The Audit Event Log is required to securely log nominated events. Sufficient memory has been allocated so events can be logged for the meter's entire lifetime.

Audit Event Log is never deleted, if size of memory for events exceeds the allocated space the device goes into out of service permanently.

The Audit Event Log can be extracted from the meter via the TTI. When extracting a hash is calculated on all logs extracted and that hash is cryptographically signed and sent to the TTI to verify the integrity and authenticity.

Event Log Structure

Each event is logged as a fixed 128 byte entry, to align with physical memory, containing:

- Index of event
- Time of event
- Event type
- Optional data (64 bytes)
- SHA256 hash of above
- Protocol version

Optional data contains information relevant to the event logged. i.e. All configuration parameters have an event logged whenever they are changed, data contains a copy of the new parameters.

Some events require multiple event entries due to needing to log more data than a single entry can contain.

Events Logged

The following is a list of the event which are logged by the meter:

- EVENT_LOG_POWER_STARTUP
- EVENT_LOG_POWER_LOSS
- EVENT_LOG_BEGIN_CHARGE_SESSION
- EVENT_LOG_END_CHARGE_SESSION
- EVENT_LOG_AFE_CONFIG_PARAM_CHANGE
- EVENT_LOG_TSOCLIENT_CONFIG_PARAM_CHANGE
- EVENT_LOG_INET_CONFIG_PARAM_CHANGE
- EVENT_LOG_PCBA_TEST_LOG_PARAM_CHANGE
- EVENT_LOG_SUBA_TEST_LOG_PARAM_CHANGE
- EVENT_LOG_HOST_PRIVATE_KEY_PARAM_CHANGE
    - The private key event is explicitly prevented from logging the private key as the optional data field for security reasons
- EVENT_LOG_TRUST_ROOT_KEY_PARAM_CHANGE
- EVENT_LOG_COMMISSION_CONFIG_PARAM_CHANGE
- EVENT_LOG_OUT_OF_SERVICE_CHARGE
- EVENT_LOG_TIME_SYNC
- EVENT_LOG_TIME_DEVIATION
- EVENT_LOG_TECHNICIAN_LOGIN
- EVENT_LOG_TECHNICIAN_LOGOUT
- EVENT_LOG_LOG_EXPORT_OCCURRED
- EVENT_LOG_FIRMWARE_UPDATE_DOWNLOAD
- EVENT_LOG_COMMISSION
- EVENT_LOG_MEAS_STORAGE_USAGE_WARN
- EVENT_LOG_RECORDS_DELETED
- EVENT_LOG_SYSCHECK_RESULT (see Self Tests)

Optional data

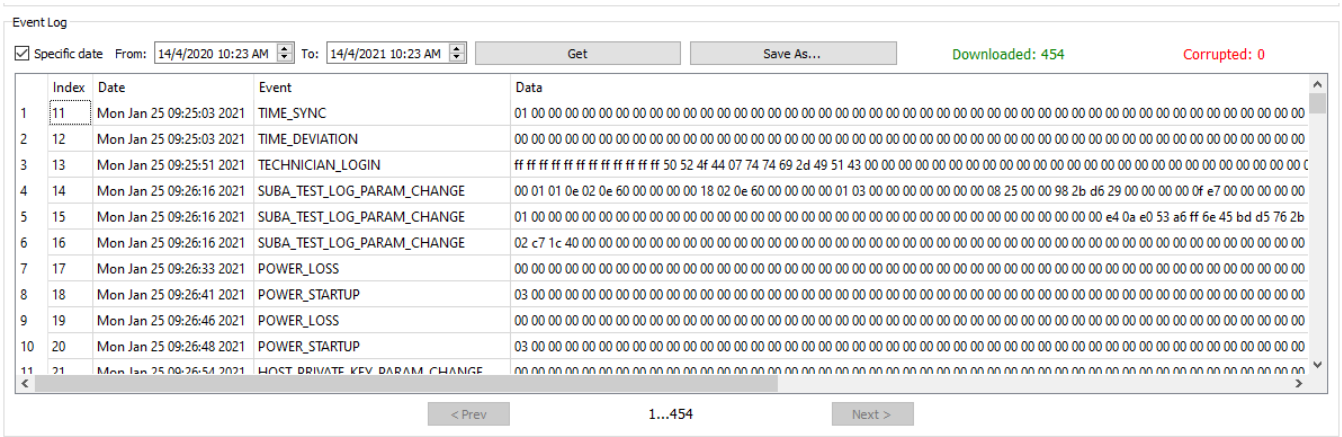
The information in optional data logged is different for different events. Only the events which include optional data logged is included:

Event type	Optional data logged	Notes
EVENT_LOG_POWER_STARTUP	Reset Cause [2 bytes]	MAP_SysCtlResetCauseGet as defined for Texas Instruments TM4C129
EVENT_LOG_BEGIN_CHARGE_SESSION	EDSI [4 bytes]	See Energy Delivery Session Identifier (EDSI)
EVENT_LOG_END_CHARGE_SESSION	EDSI [4 bytes] Charger Session ID [37 bytes]	See Energy Delivery Session Identifier (EDSI) See Supporting the SAFE Initiative

Parameter change EVENT_LOG_AFE_CONFIG_PARAM_CHANGE to EVENT_LOG_COMMISSION_CONFIG_PARAM_CHANGE Excluded: EVENT_LOG_HOST_PRIVATE_KEY_PARAM_CHANGE	Section/chunk number [1 byte] Set successfully (bool) [1 byte] Parameter data [62 bytes]	If parameter type is greater than can fit in one log, it is split into multiple sequential logs (chunks) to fit whole parameter pack. See DC Meter Configuration
EVENT_LOG_OUT_OF_SERVICE_CHARGE	OoS Fault cause [1 byte]	See Charge Session State Machine
EVENT_LOG_TIME_SYNC	hasTimeSyncSucceeded (bool) [1 byte]	
EVENT_LOG_TECHNICIAN_LOGIN EVENT_LOG_TECHNICIAN_LOGOUT	PermissionLevel (read [2 bytes]; write [2 bytes]; execute [2 bytes]; type [4 bytes]) Peer ID [32 bytes]	See Trusted Technician Interface (TTI)
EVENT_LOG_LOG_EXPORT_OCCURRED	Type Log [1 byte]	0=OCMF; 1=Event
EVENT_LOG_FIRMWARE_UPDATE_DOWNLOAD	hasDownloadSucceeded (bool) [1 byte] isServerTrustworthiness (bool) [1 byte] isImageTrustworthiness(bool) [1 byte] isBootloaderImageValid(bool) [1 byte] imageHeaderHash(bool) [32 bytes]	All bools true if successful.
EVENT_LOG_MEAS_STORAGE_USAGE_WARN	Usage Percentage [1 byte]	
EVENT_LOG_RECORDS_DELETED	Erased Block Number [2 bytes]	See External Non Volatile Storage
EVENT_LOG_SYSCHECK_RESULT	Sys check result [4 bytes]	See Self Tests

Event Log Sample

The screenshot below shows a sample of the event log available through the Trusted Technician Interface. The log can be downloaded and saved to disk using the "Save As..." button.



Once downloaded, the log file can be viewed in any plain text editor. An excerpt of a log file is provided below. Each line begins with a timestamp, lines shown are wrapping due to line length. The full contents of this log file can be viewed in A - Full Audit Event Log. The data format can be decoded according to the information described above in "Event Log Structure" and the table "Optional Data".

```
Index Date Event Data Hash Protocol version
11 Mon Jan 25 09:25:03 2021 TIME_SYNC 01 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

00 00 00 a5c7e4f3d3e78de7ded77bb52e6756913899b3defcd3cebdd7b7d47948abc37f 0
12 Mon Jan 25 09:25:03 2021 TIME_DEVIATION 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 4598e66c3d20389a1f4be83c29b7f6ee83a3937d11e6b133c2196297cc12a18e 0
13 Mon Jan 25 09:25:51 2021 TECHNICIAN_LOGIN ff ff ff ff ff ff ff ff ff ff ff ff 50 52 4f 44 07 74 74 69 2d 49
51 43 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 84f416d24ddffc080b76658a71521d3673a7155a2bc94e912565ec541a5f8479 0
14 Mon Jan 25 09:26:16 2021 SUBA_TEST_LOG_PARAM_CHANGE 00 01 01 0e 02 0e 60 00 00 00 00 18 02 0e 60 00 00 00 00
01 03 00 00 00 00 00 00 00 08 25 00 00 98 2b d6 29 00 00 00 00 0f e7 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 cb654c5a020c864aa6dc43c057e5dd977bd54e2bd483a3b61ec50c326d720a27 0
15 Mon Jan 25 09:26:16 2021 SUBA_TEST_LOG_PARAM_CHANGE 01 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 e4 0a e0 53 a6 ff 6e 45 bd d5 76 2b e2 40 3f 9f 78 c0 85 f8 03
bc 52 6e 16 84 88 7a 4d 8c1964b4a7138aebf4c2db97d9483ca1542e7a445859a3fda803f663c277b8b4 0
16 Mon Jan 25 09:26:16 2021 SUBA_TEST_LOG_PARAM_CHANGE 02 c7 1c 40 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 39990f4d77d0f1915654f785945fc2045ce0a9b3a1060404d16a8292d62d08fa 0
17 Mon Jan 25 09:26:33 2021 POWER_LOSS 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 d46b127bdbcfb3b97821dfb4b864874fd7cce3c6ac746963815995def1b0b81e 0
18 Mon Jan 25 09:26:41 2021 POWER_STARTUP 03 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 e1b162e5f38e7211666db54e2bffc142b4e34e168bb926b197d481f8a57ee966 0
19 Mon Jan 25 09:26:46 2021 POWER_LOSS 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 481d6db29195202326429e392b2c689751ea6ddac94cb8c83273df003922b240 0

Charge Session Log

Charge session information is logged with all information needed to reproduce a receipt. All variable data in the OCMF message is logged to the external non-volatile storage so on request it can be regenerated.

Every charge session uses 1 page of storage. The first 3 sectors of the page are used for OCMF entries with a hash, which are entered once the charge is completed. The last section of that page is reserved for the signature of the entries, written when OCMF message is successfully sent over MQTT. The signature of the data allows authenticity and integrity of the data to be verified.

If the OCMF message can't be sent over MQTT the signature is not written until successful. On power cycle an unsigned OCMF message will be detected, signed and sent over MQTT, once successful the signature is written to the external non-volatile storage.

The memory space reserved for charge session logs should be sufficient for more than 3 months of frequent use. Once the allocated memory space is utilised the oldest block will be erased and reused for new sessions.

The Charge Session Log can be extracted from the meter via the TTI.

Application Package Manager

- Upgrade Binary Package
- Certificate Chaining
- Update Procedure

The DC meter must have the ability to securely update legally relevant firmware while the existing application is running.

To begin a firmware update, a user must sign into the DC Meter with the Trusted Technician Interface (TTI) using a trusted certificate.

Once the firmware update is triggered, the DC Meter locks into out-of-service. The DC Meter connects to and authenticates an update server to be trustworthy. The DC Meter downloads the firmware update, verifies the integrity and the manufacturer signature.

Once the firmware download has been confirmed to be valid, the DC Meter will jump to the bootloader to install the firmware update. The bootloader verifies the downloaded firmware update and then programs as the main image.

On every startup of the device, the bootloader will calculate and verify the hash of the main image. If this hash is incorrect the main application will not be entered and the device will be inoperable.

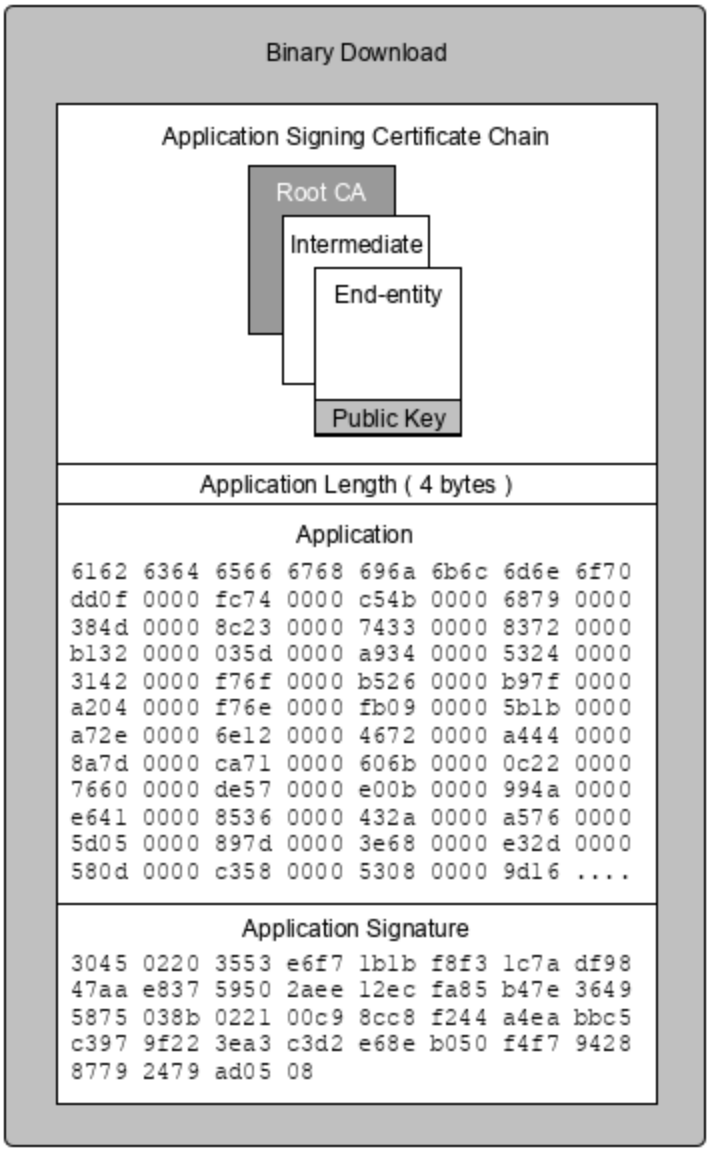
The Application Package Manager task is responsible for;

- Securely connecting to an remote Upgrade Server.
- Verifying that the Upgrade Server is trustworthy.
- Downloading a binary image from a trustworthy Upgrade Server.
- Verifying that the binary image is trustworthy and valid.

Upgrade Binary Package

The binary image contains;

- Application Signing Certification Chain
    - Used to verify the signature of the binary image.
- Application Length (in bytes)
- Application Binary (payload)
- Application Hash
- Application Signature



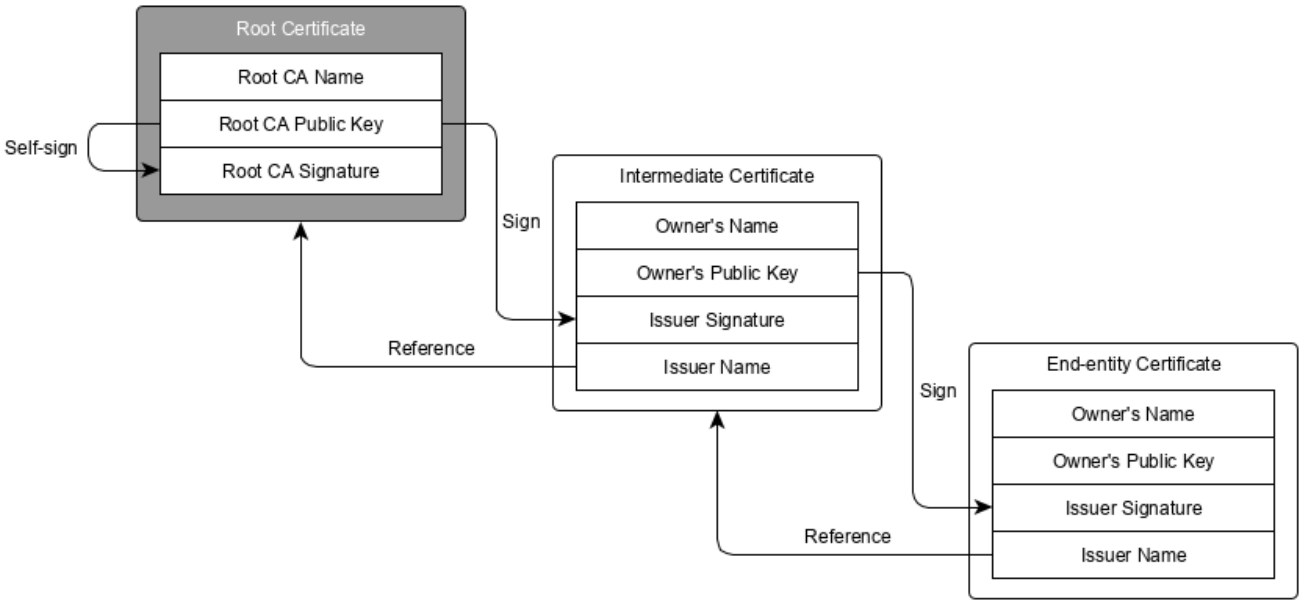
⚠ The binary image may be hosted on an Upgrade Server that is separate to the Binary Signing Server. To allow for this, the binary image is bundled with a Signature and a Signing Certificate Chain to verify the binary image trustworthiness.

Certificate Chaining

To verify the application in the binary image is trustworthy, the Signing Certificate Chain must first be verified. A certificate chain is a list of certificates followed by one or more Certificate Authority (CA) certificates (usually being self-signed).

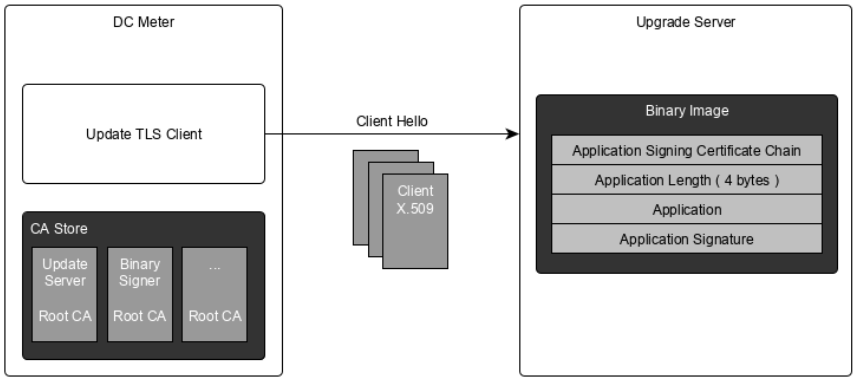
Due to resources on the meter, only a single level of certificates is supported. No intermediate certificates can exist in the chain, the end-entity must link to the root certificate. The root certificate in the list is a trust anchor and can be trusted.

Certificate chains are used in order to check that the Public Key (PK) contained in a target certificate (the first certificate in the chain) and other data contained in it effectively belongs to its subject. In order to ascertain this, the signature on the target certificate is verified by using the PK contained in the following certificate, which is the root certificate. As the last certificate is a trust anchor, successfully reaching it will prove that the target certificate can be trusted.

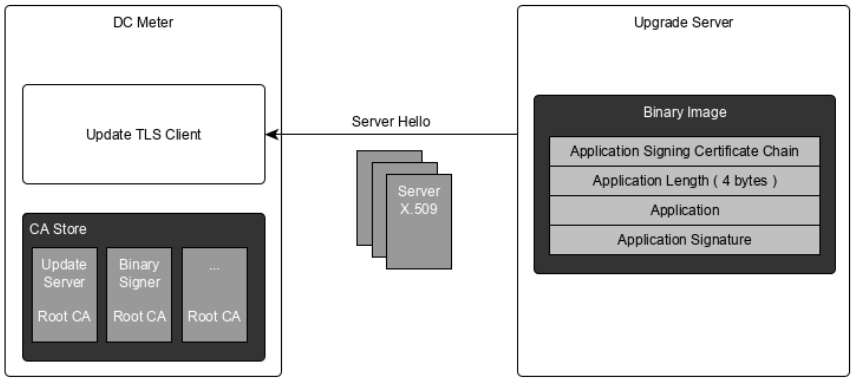


Update Procedure

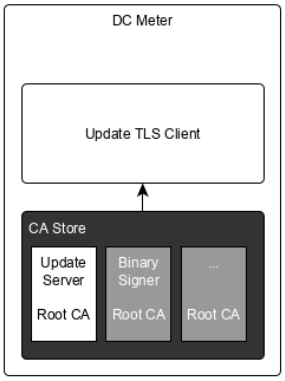
1. Connect to Update Server. Client submits certificate chain for verification.



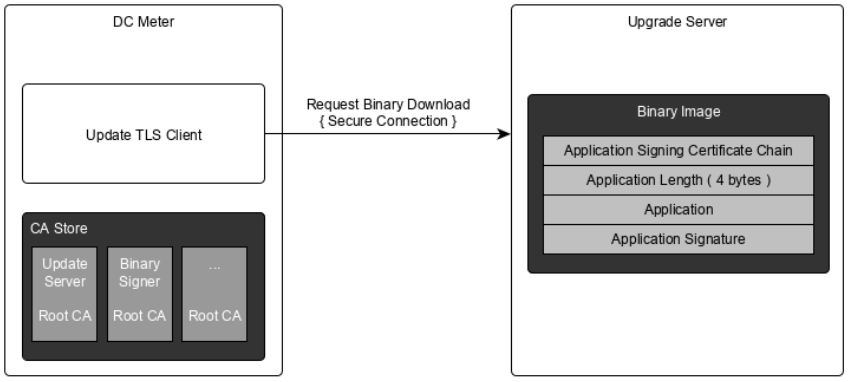
2. Server verifies Client certificate chain. Server responses with Server certificate chain.



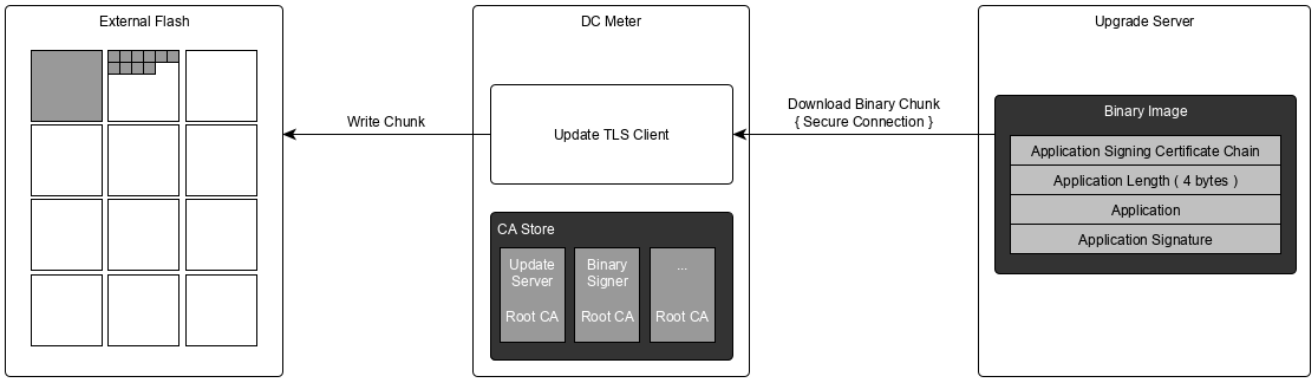
3. Client verifies Server certificate chain; using preloaded CA store as trust anchor.



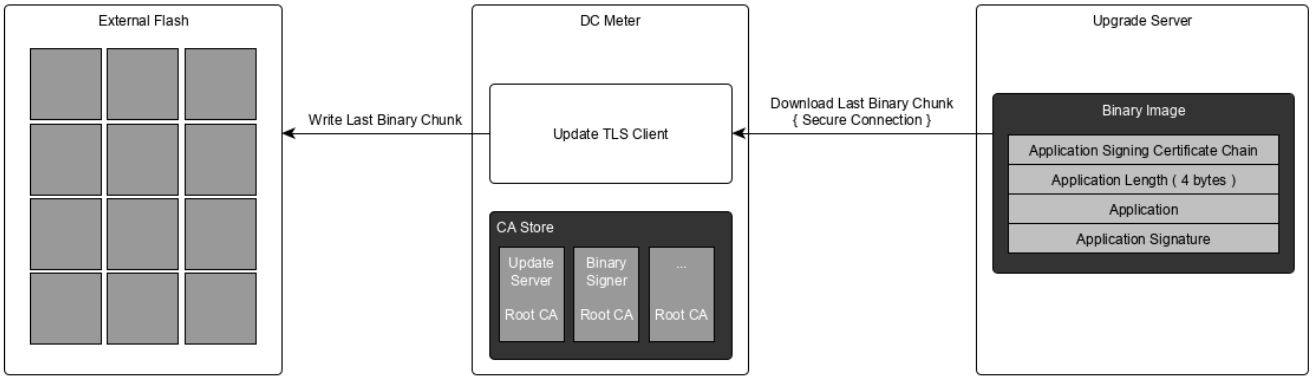
4. Client request binary download.



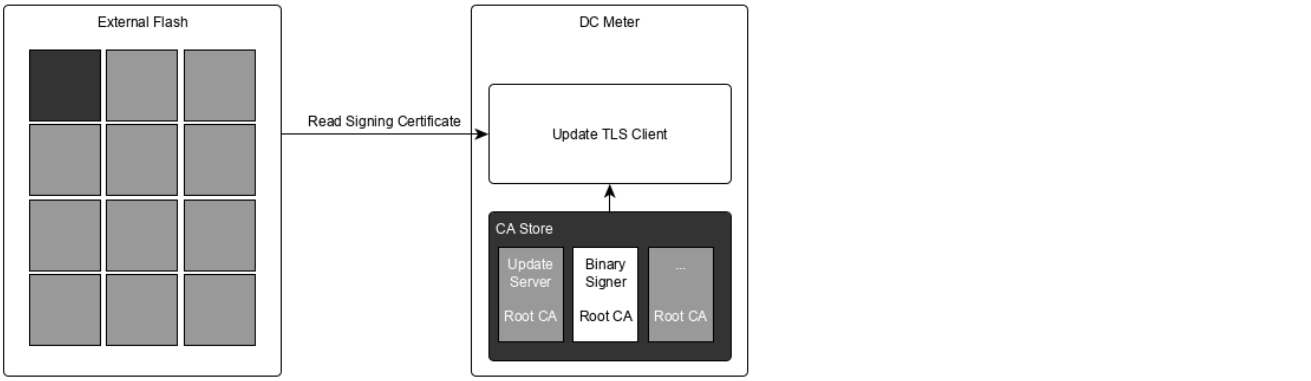
5. Client downloads binary file in chunks. Chunks are written to external flash. The SRAM isn't large enough to hold entire binary.



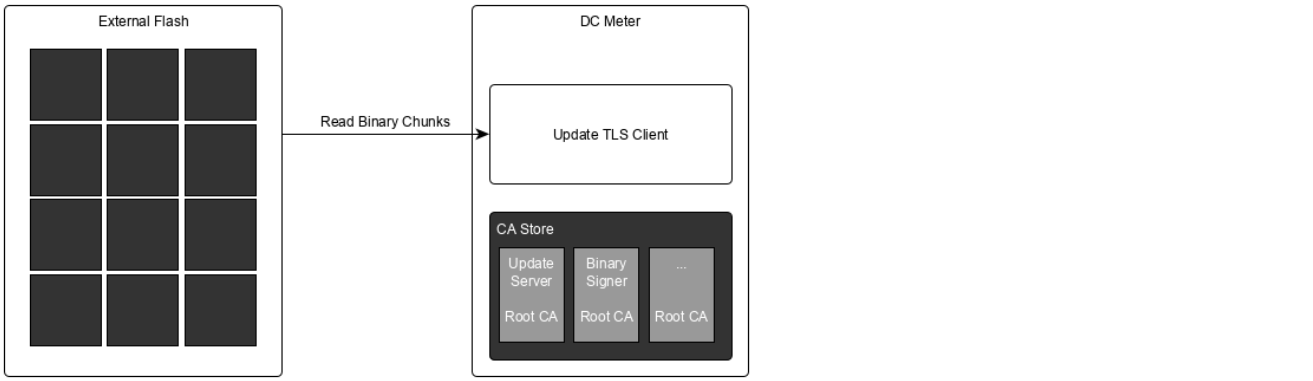
6. After last chunk is downloaded, close connection to Server.



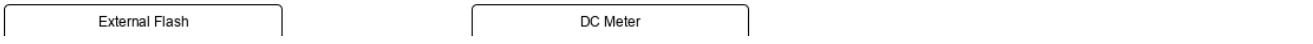
7. Read from external the first chunk containing the Binary Signing certificate chain. Verify that the Binary Signing certificate chain is trustworthy. Extract Public Key from End-Entity (EE) certificate.

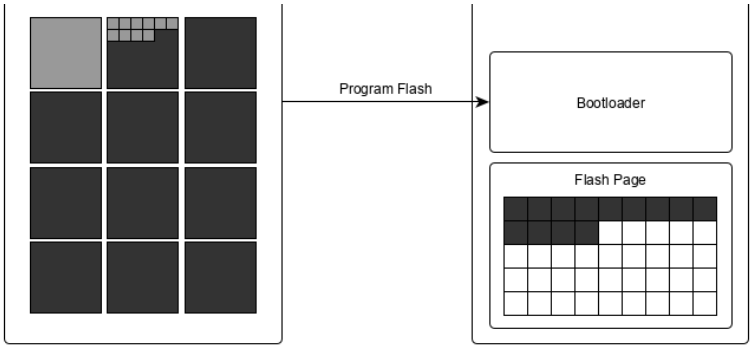


8. Sequentially compute the hash of the application binary. Use the Public Key of the EE certificate with the generated hash to verify the binary signature.



9. Application binary is verified. Jump into Bootloader and program into flash.




Program Flash
Bootloader
Flash Page

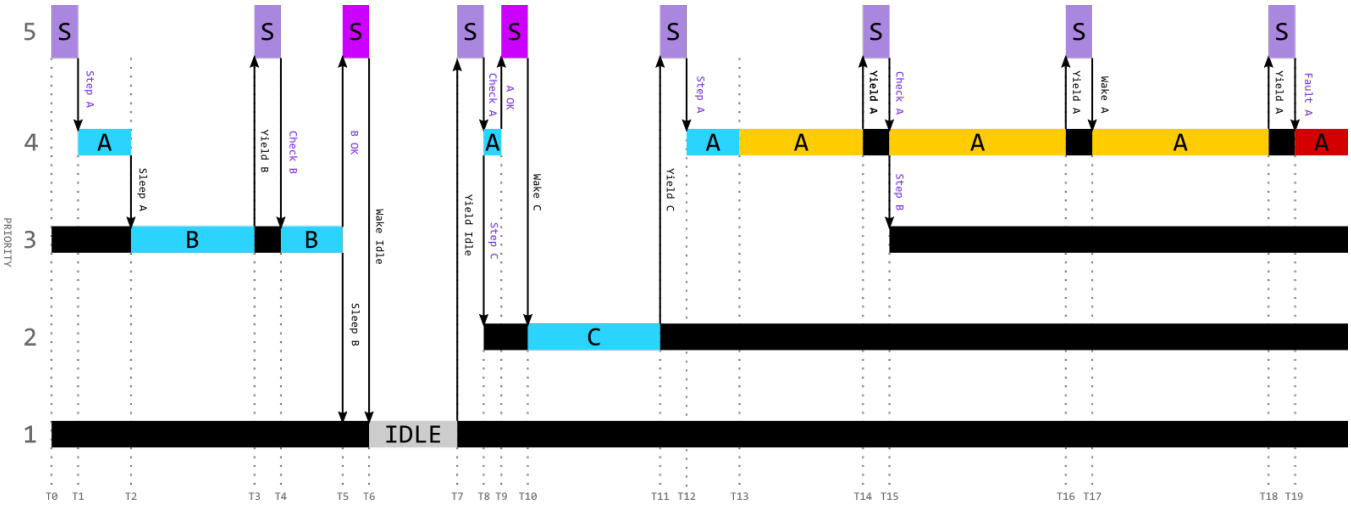
Supervisory State Machine

Introduction

The Supervisory State Machine (SSM) monitors the health of the DC Meter. The SSM resets the hardware watchdog and monitors the state of other RTOS Tasks.

Details

The SSM is the highest priority task in the DC Meter and is responsible for starting, stepping and monitoring other RTOS Tasks. The SSM is implemented using a FreeRTOS Task Notification (https://www.freertos.org/xTaskNotify.html) schema. Periodically, the SSM will issue each task with a "Health Check" Task Notification. The SSM expects that the notified task will send a "Task OK" Task Notification acknowledgement within a specified time window.



The above figure shows an example of the highest priority user task (Task A) becoming unresponsive and detected by the SSM.

Time	Description
T0 - T1	Task B blocked. SSM action; notify Task A to "step".
T1 - T2	Task A woken by "step" notification, executes, completes and sleeps.
T2 - T3	Task B unblocked and executes.
T3 - T4	Task B yields. SSM action; notify Task B to "check".
T4 - T5	Task B unblocked, executes, notifies SSM with "check" acknowledgement and sleeps.
T5 - T6	SSM woken by "check" acknowledgement notification, resets Task B watch-dog counter and sleeps.
T6 - T7	Task Idle woken, executes.
T7 - T8	Task Idle yields. SSM action; notify Task A to "check" and Task C to "step".
T8 - T9	Task A woken by "check" notification, Task A notifies SSM with "check" acknowledgement and sleeps.
T9 - T10	SSM woken by "check" acknowledgement notification, resets Task A watch-dog counter and sleeps.
T10 - T11	Task C unblocked, receives "step" notification and executes.
T11 - T12	Task C yeilds. SSM action; notify Task A to "step".
T12 - T13	Task A executes.
T13 - T14	Task A malfunctions taking CPU time and blocking tasks.
T14 - T15	Task A yields. SSM action; notify Task A to "check" and Task B to "step".
T15 - T16	Task A resumes taking CPU time and blocking tasks. Unable to receive "check" notification.
T16 - T17	Task A yields. SSM action; no action.
T17 - T18	Task A resumes taking CPU time and blocking tasks. Unable to receive "check" notification.

T18 - T19	Task A yields. SSM action; Task watch-dog counter has reached zero. Fault task.

Once a task has been identified as faulty, corrective action is for the Supervisor to software reset the entire microcontroller. This acts as a software watchdog for every individual task, whilst a hardware watchdog oversees the Supervisor task.

Self Tests

Introduction

The DC Meter must conduct self tests on critical components in order to verify the integrity of measurements and the system as a whole.

The self tests run at system start-up or during run time.

Tests during run time are either run periodically every 6 hours or on a continuous interval as required to maintain functionality.

Test	Start-up	Run Time (Periodic)	Run time (Continuous)	Causes CSSM OoS	Logged
Supply voltage - ADC			x	x	x
Supply voltage - UVLO			x	x	x
Ambient temperature			x	x	x
Shunt temperature			x	x	x
Fan state / speed			x		x
Ovenisation temperature			x	x	x
NTP last time synchronisation			x	x	Via 'RTC state' self test
NTP time variance	x	x			x
RTC backup supply voltage			x		x
RTC state	x	x		x	x
Measurement storage integrity	x	x			x
Measurement storage capacity	x		x	x	x
Configuration integrity	x			x	x
Firmware Image integrity	x			x	Bootloader image integrity via 'Firmware Update' task Operational image integrity via 'boot event' log at start of execution
AFE communications		x		x	Via 'CSSM - Out Of Service' event log if failed during charge
External Flash communications		x		x	Via 'CSSM - Out Of Service' event log if failed during charge
Entropy			x		x

Result logging

Logging of self test results is performed periodically every 6 hours.

There are a number of run time tests that independently or indirectly log an event as described in the table above.

Self Test Details

Supply voltage - ADC

The 12V supply voltage to the DC Meter is digitised using the microcontroller's internal ADC.

The measured voltage is checked to be within an acceptable range (Under and Over) for the system to operate.

Supply voltage - UVLO

The 12V supply voltage to the DC Meter is monitored by a voltage comparator circuit internal to the microcontroller.

If the supply voltage level drops below the comparators trigger level (eg: power failure), an Under Voltage Lock Out event is immediately triggered by the system.

Ambient temperature

Ambient temperature in the DC Meter enclosure is digitised using the microcontroller's internal ADC.

The measured temperature is checked to be within an acceptable range (Under and Over) for the system to operate functionally and within accuracy requirements.

Shunt temperature

The temperature of the shunt element used for accurate measurement of the DC Meter high current path is measured using a LMT01 digital sensor.

The shunt temperature is checked to be within an acceptable range (Under and Over) for the system to operate functionally and within accuracy requirements.

Fan state / speed

The fans that cool the shunt element are actively controlled based on shunt temperature. Speed feedback is provided from the fans as a train of pulses that the microcontroller monitors.

The fan control loop checks the speed of the fans to ensure they are within an acceptable range (Under and Over), when determined they should be on, for the shunt temperature to be effectively managed.

Ovenisation temperature

In order to achieve accuracy targets over the wide operating temperature range, the analogue front end circuitry is heated to a constant temperature.

The heating control is independent of the microcontroller, however, the temperature of the circuitry is digitised by the precision ADC (main ovenised component) and provided to the microcontroller.

The measured temperature is checked to be within an acceptable range (Under and Over) for the system to operate functionally and within accuracy requirements.

NTP last time synchronisation

The system time is synchronised on a regular basis to ensure it remains within the required tolerance and to detect any faults.

If a successful synchronisation has not occurred within a defined time frame, a fault is triggered as the time accuracy is not guaranteed.

NTP time variance

The system time is synchronised on a regular basis using the Simple Network Time Protocol to ensure it remains within the required tolerance and to detect any faults.

After a successful synchronisation, the change in time is checked to be within an acceptable range (Under and Over) to ensure the system was within accuracy requirements.

RTC backup supply voltage

In order to preserve system time keeping during a power outage an on board backup energy source is used. The voltage is monitored by a voltage comparator circuit internal to the time keeping module in microcontroller.

If the voltage level drops below the comparators trigger level, an under voltage event is determined implying that time keeping is not guaranteed if the main supply fails.

RTC state

The Real Time Clock is responsible for time keeping on the DC Meter. It is a module internal to the microcontroller.

The configuration of the time keeping module and the present value of time is monitored to ensure the system is operating correctly.

Measurement storage integrity

All power measurements (charge sessions) and events are logged to non volatile memory. All data saved to memory is stored with error detecting information.

The integrity of all saved data is verified to ensure full traceability is preserved.

Measurement storage capacity

The external non volatile memory has a finite capacity which is partitioned to store different types of data.

The remaining storage capacity for event logs is checked against several thresholds in order to help avoid the impact of completely running out of storage capacity.

Configuration integrity

All configuration items are stored in non volatile memory along with error detecting information.

The integrity of all saved configuration is verified to ensure the system will operate functionally and within accuracy requirements.

Firmware Image integrity

The DC Meter firmware is separated into two main functional blocks, the bootloader and operational (main DC Meter logic) firmware. All firmware includes error detecting information to verify its integrity.

The bootloader is run at first power up and during the final stage of a firmware update. The bootloader checks the operational firmware integrity before it is executed.

The operational firmware checks the integrity of the bootloader before a firmware update process is started.

AFE communications

Metrological measurements are performed by an external ADC which are transferred to the microcontroller through an isolated bi-directional communications link.

If new measurements are not received by the microcontroller within a specified time period, an error is determined implying that communications or the ADC have failed.

External Flash communications

Flash memory external to the microcontroller is used for all logging and storage of firmware updates. This test only covers the communications link operation.

Any failure to access the external flash memory triggers an error which will prevent further charge sessions from being metered.

Entropy

The sources of random data are checked and assured to be within entropy limits.

DC Meter Configuration

The configuration of the DC meter is separated into blocks and saved in the internal EEPROM. All configuration is written through the TTI. Each configuration block can be given access to read and/or write independently of other blocks.

On every change of EEPROM block, an event is added to the audit log. All blocks are broken into 128 byte sections (or multiples of) and contains a hash verifying the integrity of the data. Any data that has been determined to be corrupted (i.e. hash verification failed) will not be used and flagged as an error.

- AFE (Calibration) Configuration
- TSO Configuration
- Inet Configuration
- Commissioning Configuration
- Configuration Summary
- Other EEPROM Data

AFE (Calibration) Configuration

This will be written during production in controlled conditions.

This accounts for the individual unit's manufacturing tolerances, and is critical to the metrological accuracy. It contains:

- Current offset (nA) for low gain
- Current offset (nA) for high gain
- Current scale factor (ppm) for low gain
- Current scale factor (ppm) for high gain
- Voltage offset (nV) for channel 1
- Voltage offset (nV) for channel 2
- Voltage scale factor (ppm) for channel 1
- Voltage scale factor (ppm) for channel 2
- Cable resistance (uohm) for channel 1
- Cable resistance (uohm) for channel 2

All parameters are validated that they are within a correct range before they are written. i.e. scale must be 90-110%

TSO Configuration

This will be written during production in controlled conditions.

This is used to configure the parameters need to connect to the TSO, and allow the same DC Meter firmware to support multiple screens. It contains:

- Number of characters in X and Y direction
- Number of pixels in overlay in X and Y direction
- Location of origin of overlay in X and Y pixels
- Location of start of character field in overlay in X and Y pixels
- Border thickness in pixels
- Screen rotation
- Colour map
- Video source: FPGA or HMI
- Selection of screen type: ISD or PK

Parameters for the TSO is validated on the TSO itself.

Inet Configuration

This will be written during production in controlled conditions.

This is used to configure the parameters need to connect to the configurable network interfaces, It contains:

- DNS primary - Resolves network names to IP address, first attempt
- DNS secondary - Resolves network names to IP address, second attempt
- UDP multicast - IP address for debug output
- UDP multicast - port for debug output
- NTP primary - web address for time server
- MQTT broker - web address or IP to connect to
- Update server - web address or IP to connect to download file for boot loading

- Update file path - file name for boot loading
- Static IP address - IP address of the meter to be once commissioned
- Static IP net mask - net mask of the meter to be once commissioned
- Static IP gateway - gateway address for meter to use once commissioned

Commissioning Configuration

This will be written during production in controlled conditions.

This is used to configure the commissioning parameters needed, it contains:

- Commissioned flag - set when all tests have passed in production
- Serial number of device
- 2 x Tamper seal IDs
- CPO public key - used to verify sent tariffs
- Default tariff name used on startup
- Parameters that define the time zone rules applied

The commissioned flag and the time zone parameters are checked to ensure they are in the correct range before they are written.

Configuration Summary

The following table summarises the configuration items specified above.

Subsection	Configuration Item	Type/Unit	Legally Relevant?	Changes Logged?
AFE (Calibration)	Current offset, low gain	nA	Yes	Yes
	Current offset, high gain	nA	Yes	Yes
	Current scale, low gain	ppm	Yes	Yes
	Current scale, high gain	ppm	Yes	Yes
	Voltage offset, channel 1	nV	Yes	Yes
	Voltage offset, channel 2	nV	Yes	Yes
	Voltage scale, channel 1	ppm	Yes	Yes
	Voltage scale, channel 2	ppm	Yes	Yes
	Cable resistance, channel 1	uOhm	Yes	Yes
	Cable resistance, channel 2	uOhm	Yes	Yes
TSO	No. characters X	Number	No	Yes
	No. characters Y	Number	No	Yes
	No. overlay pixels X	Pixels	No	Yes
	No. overlay pixels Y	Pixels	No	Yes
	Overlay origin X	Pixels	No	Yes
	Overlay origin Y	Pixels	No	Yes
	Start character field X	Pixels	No	Yes
	Start character field Y	Pixels	No	Yes
	Border thickness	Pixels	No	Yes
	Screen rotation	Enum	No	Yes
	Colour map	Dict	No	Yes
	Video source	Enum	No	Yes
	Screen type	Enum	No	Yes
Inet	DNS primary	IP	No	Yes
	DNS secondary	IP	No	Yes
	UDP multicast address	IP	No	Yes
	UDP multicast port	Number	No	Yes
	NTP primary	url	No	Yes
	MQTT broker	IP/url	No	Yes

	Update server	IP/url	No	Yes
	Update file path	String	No	Yes
	Static IP	IP	No	Yes
	Static IP net mask	IP	No	Yes
	Static IP gateway	IP	No	Yes
Comissioning	Comissioned Flag	Bool	Yes	Yes
	Serial Number	Number	Yes	Yes
	Tamper Seal ID 1	Number	Yes	Yes
	Tamper Seal ID 2	Number	Yes	Yes
	CPO public key	-	Yes	Yes
	Default tarriff name	String	Yes	Yes
	Time Zone config	Dict	No	Yes

Other EEPROM Data

There is some data stored in EEPROM that is not configuration or set via the TTI:

PCBA Test

This will be indirectly written during outsourced production.

It is used to verify the hardware functionality of the board and the result is checked before commissioning.

Through the TTI, when the PCBAs are in a dedicated jig, this test is initiated and automatically checked whether results are in range.

It checks numerous hardware elements on the board.

The EEPROM contains parameters which are checked by the firmware and given a pass/fail result.

Sub-assembly Test

This will be indirectly written during outsourced production.

It is used to verify the hardware functionality of the DC Meter and the result is checked before commissioning.

Through the TTI, when the DC Meter is assembled, this test is initiated and automatically checked whether results are in range.

It checks the fan speed and shunt temperature.

The EEPROM contains parameters which are checked by the firmware and given a pass/fail result.

Private Key Data

This will be written automatically during production after communicating with the Trust Anchor.

Private key is never displayed, exported, or read externally from the DC Meter at any time but is saved to internal EEPROM. A new private key may be generated, but never seen. The parameter data change for a key generated is not logged to the audit log for security reasons.

Trust Root Key Data

This will be written automatically during production whilst communicating with the Trust Anchor.

It is used to prove the identity of the meter and verify the identity of authentic TSOs. It contains:

- Trust root public key
- Device public key
- Signature of above by the trust root

Time on the DC Meter

Overview

Time and timekeeping on the DC Meter is critical to a number of functions.

Time is kept by an internal accurate RTC with external crystal. Real calendar time (UTC) is obtained by syncing to an NTP server on startup and every 6 hours when not charging.

The accuracy of time passing is critical to the overall metrological accuracy, but is not affected by accuracy to real calendar time.

UTC time is used to timestamp logs for audit events and charge sessions. Time is also displayed to the user on the TSO, corrected for local time.

Validity Period

The onboard crystal's accuracy ensures that the DC Meter time will be within the acceptable tolerance if an NTP sync was successful in the last 30 days.

The onboard super capacitor powers the RTC even when external power has been removed, so time remains valid as long as power is restored within 7 days.

Timezone

To reliably display local time on the TSO, time zones must be accounted for including daylight savings.

The time zone rules are used to determine what the present offset from UTC is in effect.

Time zone rules can be configured so any timezone can be accounted for. 2 rules are configured, standard time and daylight savings time.

Time zone configuration parameters for each rule include:

- Time zone abbreviation
- Hour/Minute offset from UTC
- Type - standard/daylight savings
- Ordinal day rule applies - first/last/always
- Weekday which rule transitions
- Month which rule transitions
- Hour/Minute which rule transitions
- Time Reference when rule transitions - UTC or local

Trusted Technician Interface (TTI)

Overview

The Trusted Technician Interface (TTI) is a tool used to configure, test and commission the DC Meter.

This functionality forms a command interpreter which recognises valid commands and rejects invalid ones.

The TTI has two parts, the embedded firmware running on the meter and the GUI application running on a PC which communicate over TLS 1.2. The meter interacts with the GUI using a defined interface and the meter validates any commands sent to it. The GUI is not considered legally relevant software.

The TTI will authenticate the allowable commands by using extensions within an x.509 certificate. These extensions can be used to limit access to a reduced set of commands,

Certificate Usage

The x.509 certificates passed to the TTI will be verified using a root CA pre-loaded into the image on the DC Meter. A valid certificate must be provided for the TTI to allow access.

By using certificate extensions, access can be allowed or restricted to any of the valid commands. By further use of extensions, two types of certificates will be created: production and customer. Production certificates will only be accepted by the DC Meter if it is not yet commissioned.

Customer certificates will only ever be allowed to be generated as short lived (expiry in hours) and with reduced permissions. This reduces the security risk of a certificate being copied and being obtained by unauthorised personnel.

Production certificates will be long lived, with all permissions but should not be taken outside production facilities. By marking as production certificate it reduces the security risk of being copied and obtained by unauthorised personnel as it will not be valid for units in the field.

X.509 Certificate Extensions

Certificate extensions are embedded into the X509 certificate when the certificate is first generated. Certificate extension may be viewed, along with other certificate properties. Certificate extensions are not secret. Editing a certificate extension will invalidate the certificate signature, and thus render the certificate unusable.

In order to provide the granularity required for the TTI, a methodology called Capability Security Model is being used. It is based on the following principles:

1. There are no user groups or roles. Each "user" is assigned their capabilities individually.
2. There are no stored lists of users.
3. Authentication is granted for each user based on X.509 certificates, signed by Tritium (or a Tritium approved CA). The DC Meter has a preloaded CA and how to check X.509 certificates with them.
4. Capabilities are a combination of:
    a. permitted actions (read/write/execute) and
    b. services to perform the actions on.
5. To ensure capabilities are not shared to people they shouldn't be, the user privileges form part of the certificate.

The figure below (from https://knowledge.digicert.com/solution/SO16297.html) summarises how certificate chaining works.

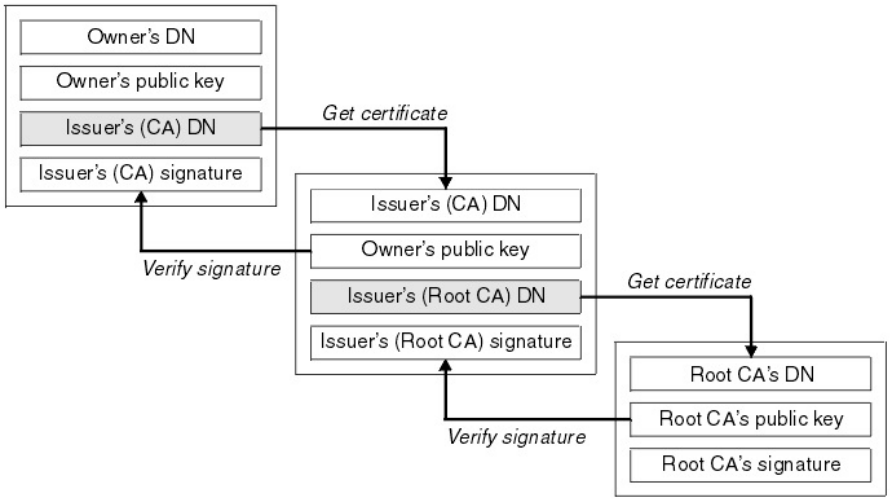


Figure – Certificate chaining example

Commands

The complete list of allowable commands for the TTI interface are as below:

Command	Purpose	Type (rwx)	Mask Offset (for cert gen)
TLSSERVER_COMMAND_GET_TELEMETRY	Gets all telemetry information from DC Meter	r	0
TLSSERVER_COMMAND_GET_CONFIG_AFE	Gets the configuration parameters of the AFE	r	1
TLSSERVER_COMMAND_GET_CONFIG_TSOIF	Gets the configuration parameters of the TSO	r	2
TLSSERVER_COMMAND_GET_CONFIG_INET	Gets the configuration parameters of the Inet	r	3
TLSSERVER_COMMAND_GET_PCB_ASSEMBLY_TEST_LOG	Gets the results of the PCBA test	r	4
TLSSERVER_COMMAND_GET_SUB_ASSEMBLY_TEST_LOG	Gets the results of the Sub-assembly test	r	5
TLSSERVER_COMMAND_GET_OCMF_LOG_INIT	Starts process to get OCMF logs	r	6
TLSSERVER_COMMAND_GET_OCMF_LOG	Gets the OCMF logs	r	7
TLSSERVER_COMMAND_GET_EVENT_LOG_INIT	Starts process to get event logs	r	8
TLSSERVER_COMMAND_GET_EVENT_LOG	Gets the event logs	r	9
TLSSERVER_COMMAND_GET_EVENT_LOG_SIGNATURE	Gets the event log signature	r	10
TLSSERVER_COMMAND_GET_CONFIG_COMMISSION	Gets the configuration parameters of the commissioning	r	11
TLSSERVER_COMMAND_SET_CONFIG_AFE	Sets the configuration parameters of the AFE	w	1
TLSSERVER_COMMAND_SET_CONFIG_TSOIF	Sets the configuration parameters of the TSO	w	2
TLSSERVER_COMMAND_SET_CONFIG_INET	Sets the configuration parameters of the Inet	w	3
TLSSERVER_COMMAND_SET_CONFIG_COMMISSION	Sets the configuration parameters of the commissioning	w	4
TLSSERVER_COMMAND_START_PCB_ASSEMBLY_TEST	Starts the automated PCBA test	x	0
TLSSERVER_COMMAND_START_SUB_ASSEMBLY_TEST	Starts the automated subassembly test	x	1
TLSSERVER_COMMAND_GET_PCB_ASSEMBLY_TEST_STATE	Gets the current status of the PCBA test	x	2
TLSSERVER_COMMAND_GET_SUB_ASSEMBLY_TEST_STATE	Gets the current status of the Subassembly test	x	3
TLSSERVER_COMMAND_UPDATE_START	Starts the automated firmware update process	x	4
TLSSERVER_COMMAND_UPDATE_GET_STATUS	Gets the current status of the firmware update process	x	5

GUI

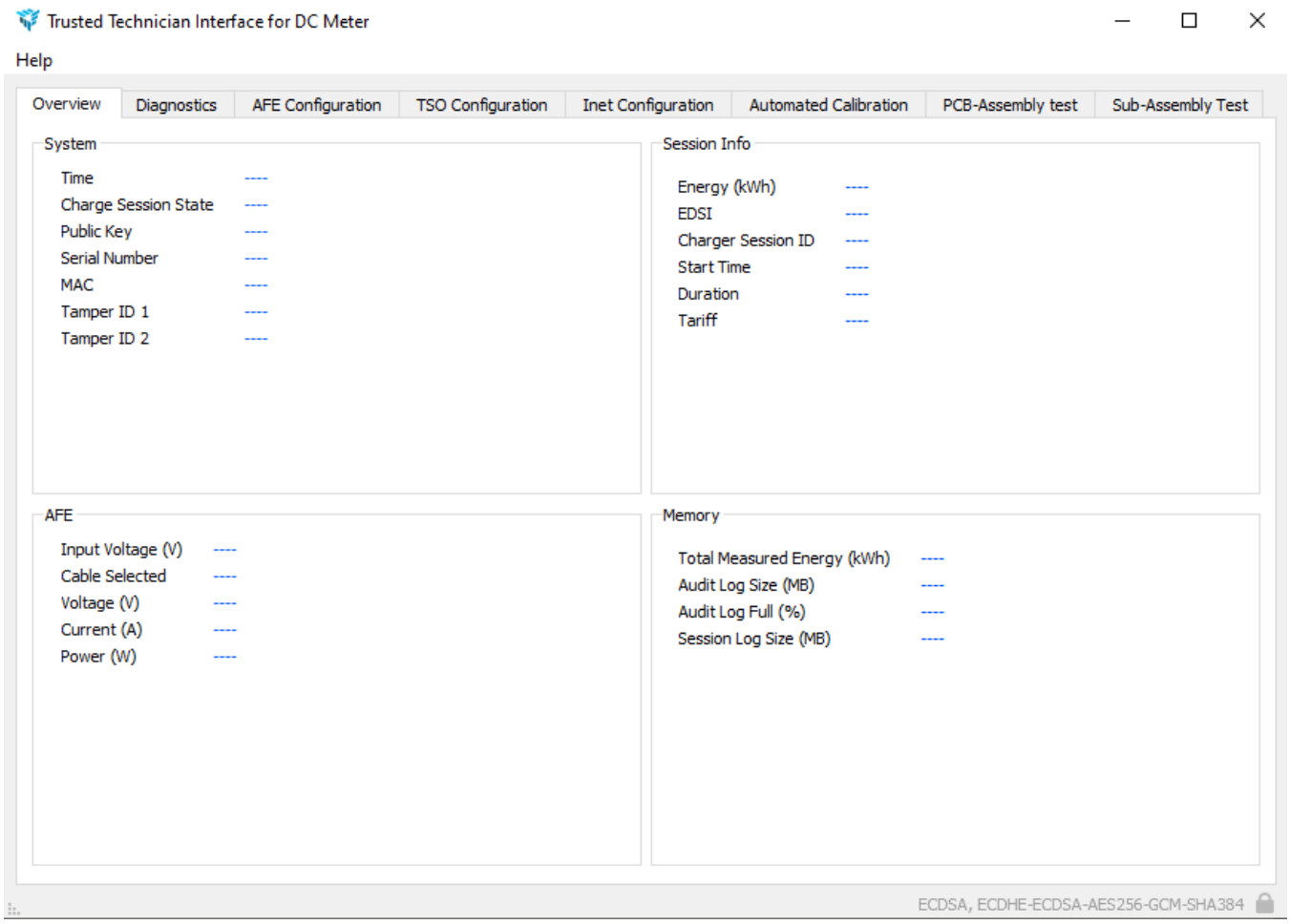
The TTI GUI provides an interface for a user to easily access information from the DC Meter. TTI GUI will dynamically allocate menu based on the certificate permissions provided.

Tabs' Purpose

- Overview and Diagnostics - provide telemetry data
- AFE, TSO, Inet, Commissioning Configuration - all allow configuration to be read and written
- Automated Calibration - allows sequence to be run in a production jig to calibrate the meter
- PCB/Sub Assembly Test - allows sequence to be run in a production jig to verify hardware build of the meter
- Logs - allows extraction of charge session and audit event logs

TTI Screenshots:

Overview:



Diagnostics:

Overview | Diagnostics | AFE Configuration | TSO Configuration | Inet Configuration | Automated Calibration | PCB-Assembly test | Sub-Assembly Test

Task Manager

Heap

Free Size	60304
Minimum Ever Free Size	53168

Tasks

Name	CPU Load	Stack
IDLE	94.7	231
tlsServer	2.8	128
mqttClient	1.0	156
cssm	0.9	404
lwipCore	0.2	762
SAFEMsgTask	0.2	335
demoTask	0.1	108
lwipEmac	0.0	206
udpClient	0.0	80
updateClient	0.0	463
tsoif	0.0	190
ntpClient	0.0	38
assemblyTest	0.0	349
timerService	0.0	100

Thermal

Shunt Temp (°C)	34.4
Amb Temp (°C)	41.1
AFE Temp (°C)	70.0
Fan 1 State	0
Fan 2 State	0
Fan 1 RPM	0
Fan 2 RPM	0

Build Information

Git Branch	TDM-2253
Git Hash	eb5e60eac7e91b6a4b9c1e352d887e920485a6b8
Build Date	Apr 17 2020
Build Time	09:12:58
Application Hash	750c2fe9d7ce7970c534c3d2b43ad012d2673c604e1db39fd41dd159ea20cc4c
Firmware Version	1.005